

A wide-angle photograph of the Stanford University campus. In the background, the tall, white, neoclassical tower of the Hoover Tower with its red-tiled dome stands prominently against a clear blue sky. To the right, a large, multi-story stone building with a red-tiled roof and numerous windows is visible. The foreground features a green lawn, a paved path, and several people walking. A decorative fountain with multiple tiers and statues is situated in the middle ground. The overall scene is bright and sunny, with lush green trees and manicured lawns.

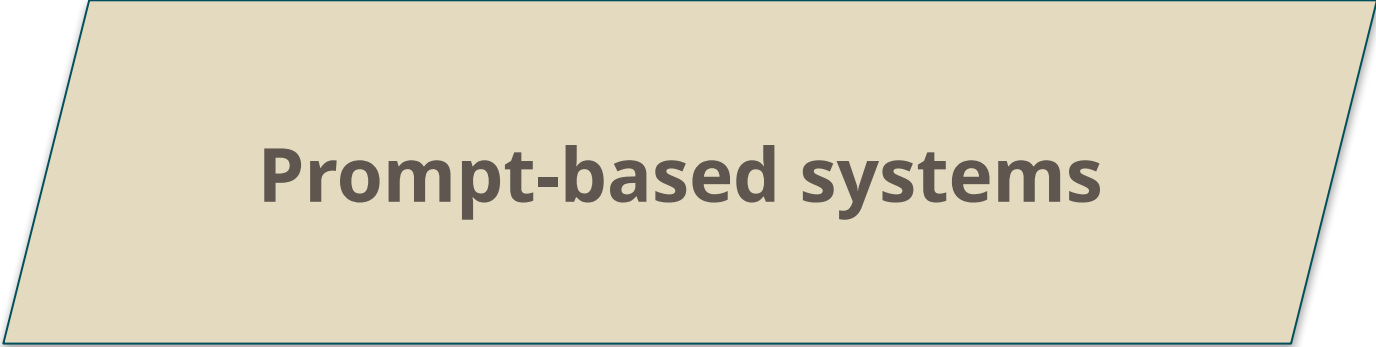
Evaluation and Security of AI-based Systems

John C Mitchell
Stanford University

Advances in AI system architectures, evaluation, and construction

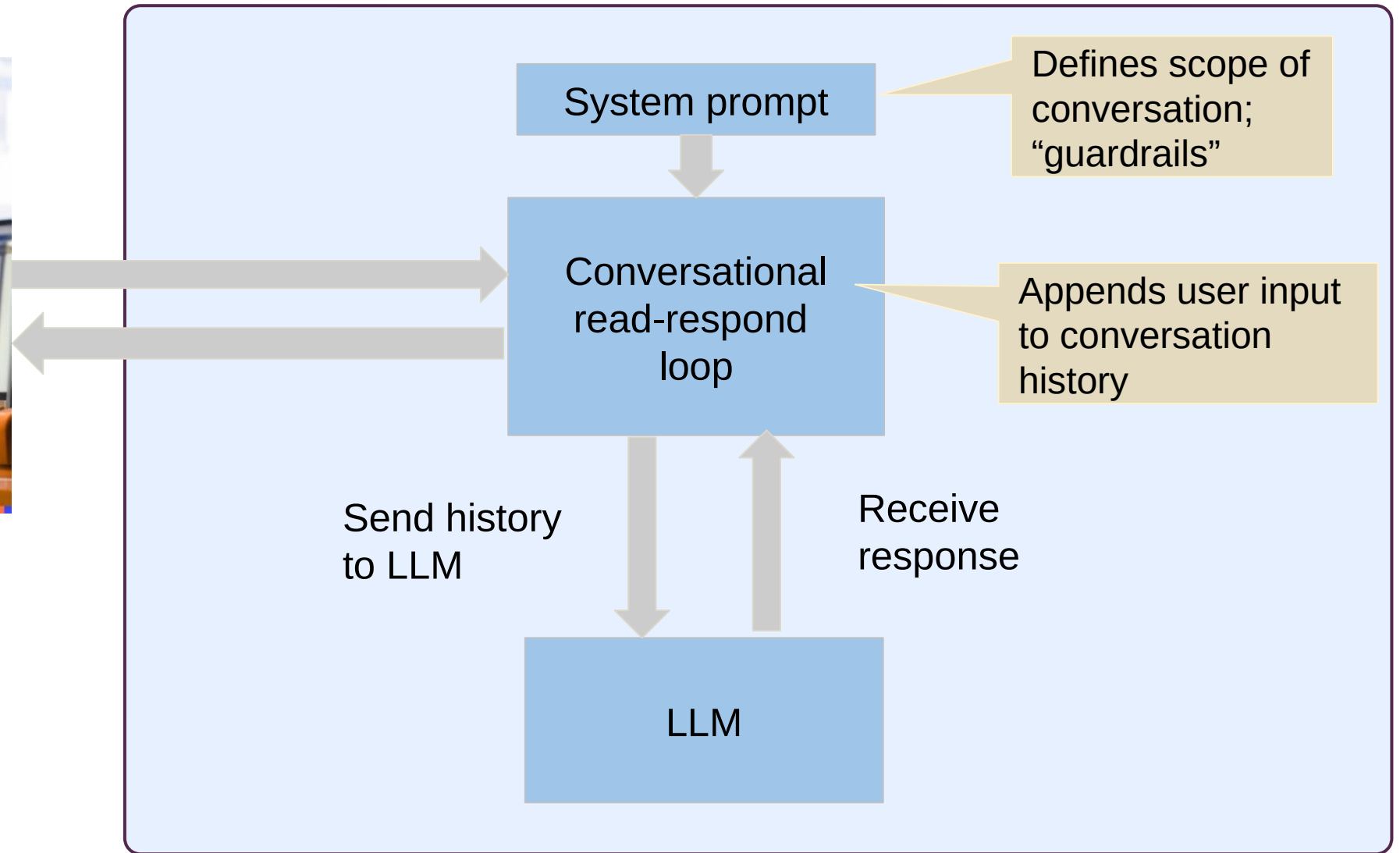
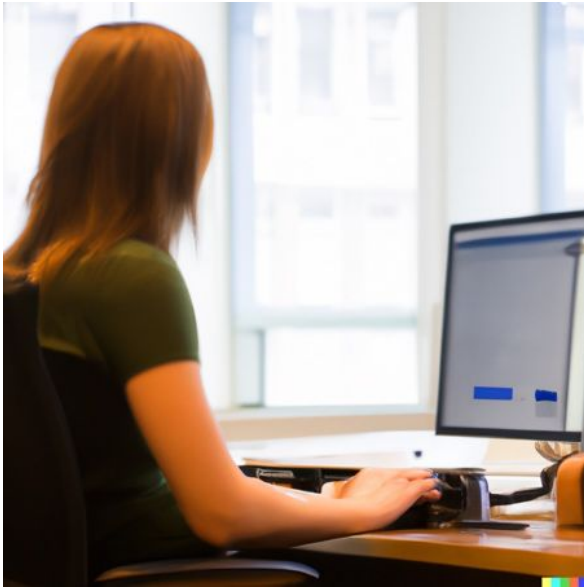
Four sessions this week

- 
- 1) Introduction to AI systems evaluation and security
 - 2) Advancing architectures of AI-based systems
 - a) Architecture and evaluation for: prompting, RAG, agentic AI systems
 - b) Prompt optimization and vibe coding
 - 3) Security for AI-based systems (part 1)
 - a) Direct and indirect prompt injection
 - b) Control flow hijacking
 - 4) Security for AI-based systems (part 2)
 - a) Agent meltdown
 - b) Defenses

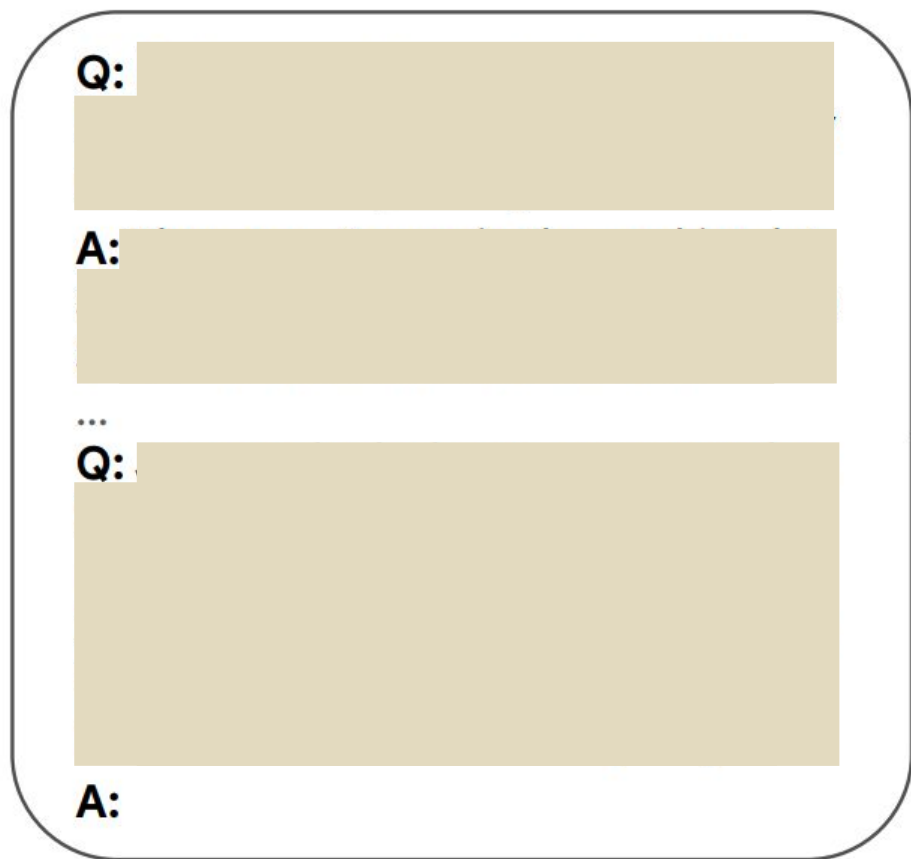


Prompt-based systems

Prompt-based system (chatbot, therapist, tutor,...)



Chain-of-Thought Prompting



Show example question
with
chain-of-thought
response



Follow with desired
question

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., ... & Zhou, D. (2022). Self-consistency improves chain of thought reasoning in language models. arXiv preprint arXiv:2203.11171.

Chain-of-Thought Prompting

Q: *If there are 3 cars in the parking lot and 2 more cars arrive, how many cars are in the parking lot?*

A: There are 3 cars in the parking lot already. 2 more arrive. Now there are $3 + 2 = 5$ cars. The answer is 5.

...

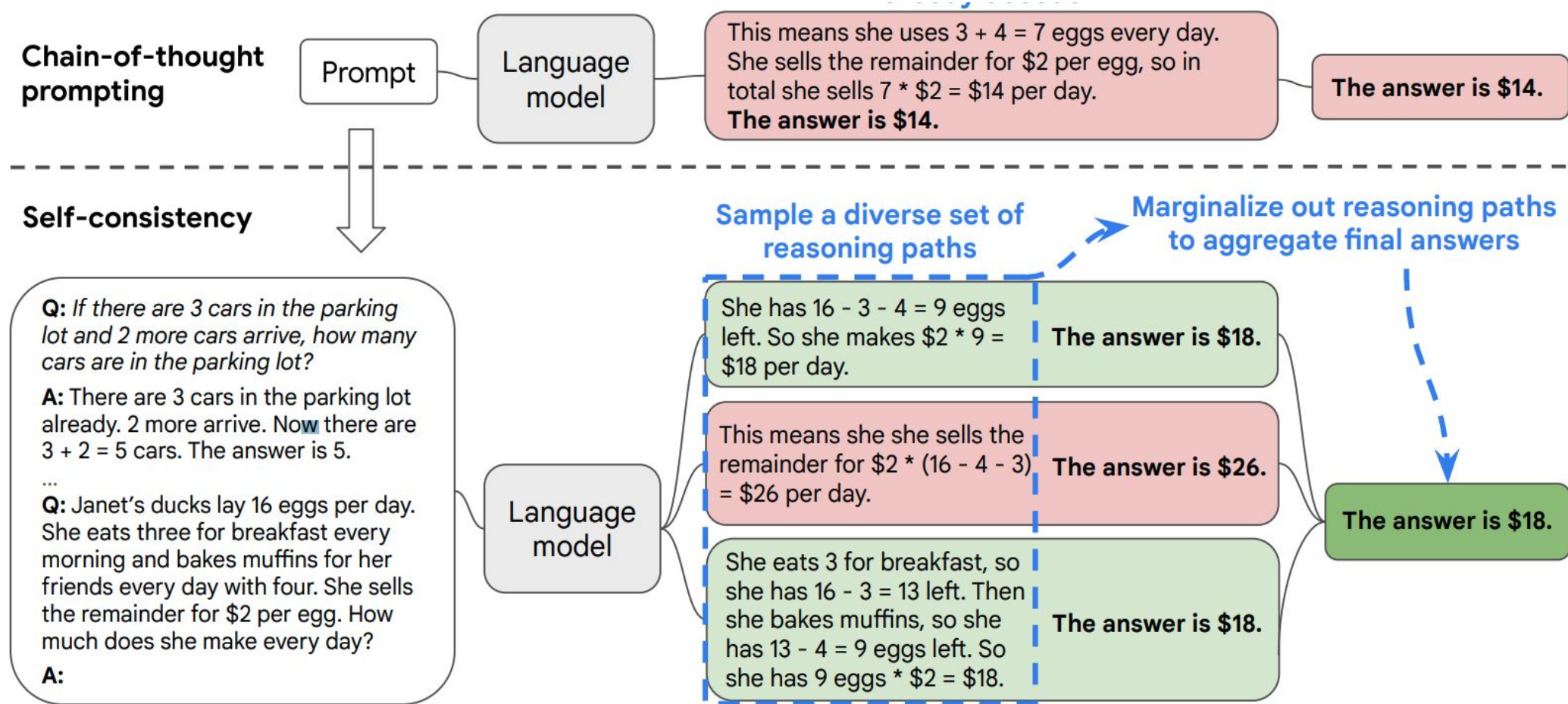
Q: Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder for \$2 per egg. How much does she make every day?

A:

Show example question with chain-of-thought response

Follow with desired question

Self-Consistency Reasoning



Subquestion Reasoning

Stage 1: Decompose Question into Subquestions

Q: It takes Amy 4 minutes to climb to the top of a slide. It takes her 1 minute to slide down. The water slide closes in 15 minutes. How many times can she slide before it closes?

Language Model

A: To solve "How many times can she slide before it closes?", we need to first solve: "How long does each trip take?"

Stage 2: Sequentially Solve Subquestions

Subquestion 1

It takes Amy 4 minutes to climb to the top of a slide. It takes her 1 minute to slide down. The slide closes in 15 minutes.

Q: How long does each trip take?

Language Model

A: It takes Amy 4 minutes to climb and 1 minute to slide down. $4 + 1 = 5$. So each trip takes 5 minutes.

Append model answer to Subquestion 1

It takes Amy 4 minutes to climb to the top of a slide. It takes her 1 minute to slide down. The slide closes in 15 minutes.

Q: How long does each trip take?

A: It takes Amy 4 minutes to climb and 1 minute to slide down. $4 + 1 = 5$. So each trip takes 5 minutes.

Subquestion 2

Q: How many times can she slide before it closes?

Language Model

A: The water slide closes in 15 minutes. Each trip takes 5 minutes. So Amy can slide $15 \div 5 = 3$ times before it closes.

Trust Considerations for Prompting

- **Untrusted user input is passed to a model**
 - Jailbreaking: subvert restrictions added by application
- **Model output is returned to the user**
 - Can leak information, compromise privacy
 - Difficult to ensure alignment
 - Bias, Cultural sensitivity, ...

GENERAL MOTORS

Chevy dealership's AI chatbot suggests Ford F-150 when asked for best truck



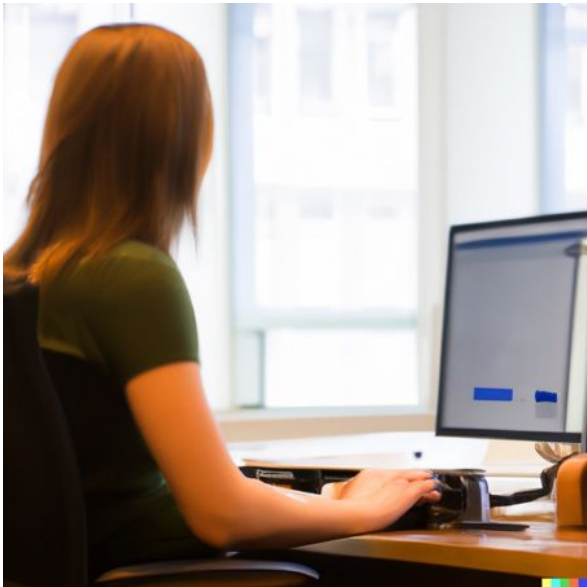
Phoebe Wall Howard

Detroit Free Press

Dec. 19, 2023 | Updated Dec. 20, 2023, 6:46 a.m. ET

Retrieval-Augmented Generation (RAG)

Retrieval-Augmented-Generation (RAG)



Retrieve via vector search or map prompt to a query and perform traditional search

1: Retrieve

Grounding Corpus

results

Retrieve Augment
Generate (RAG)

2: Augment

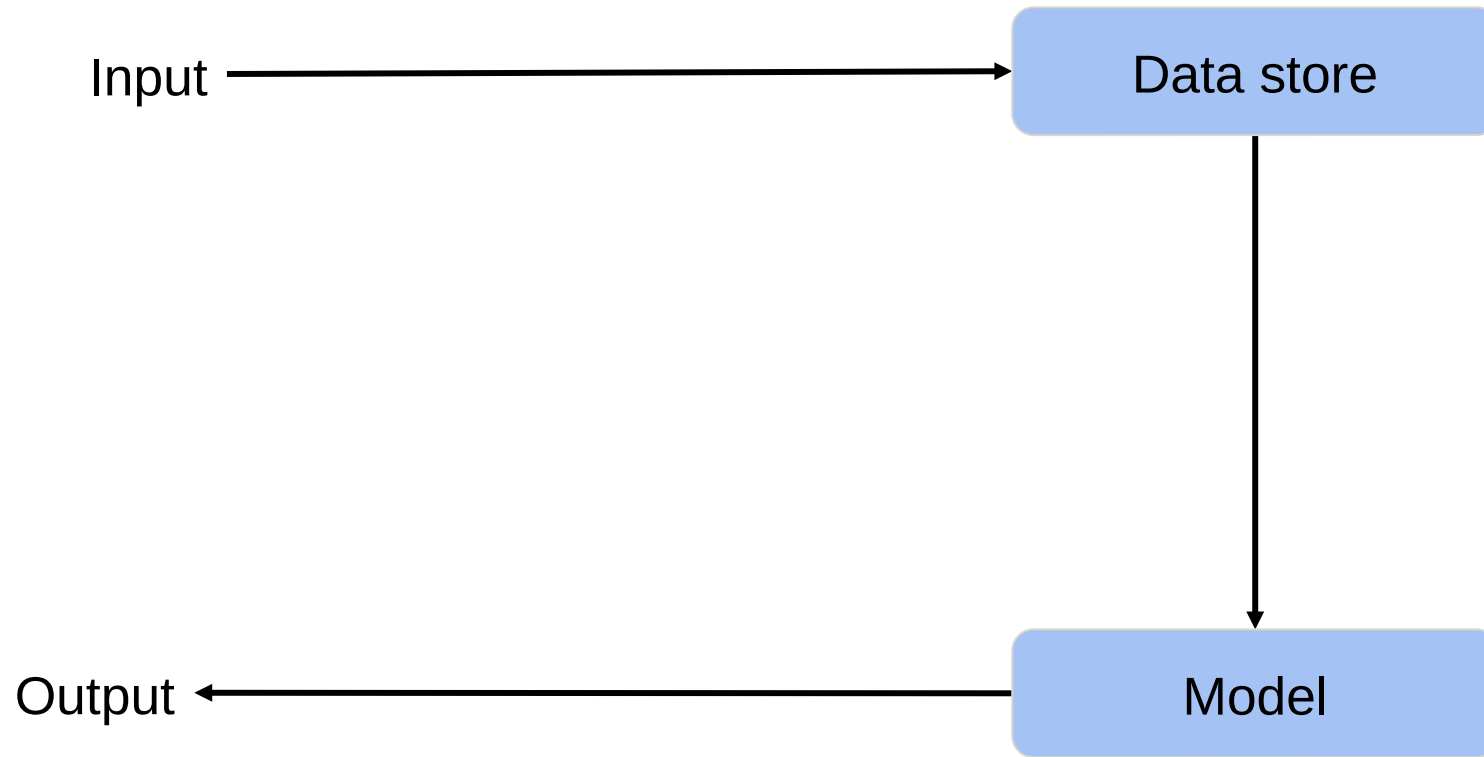
3: Generate

LLM

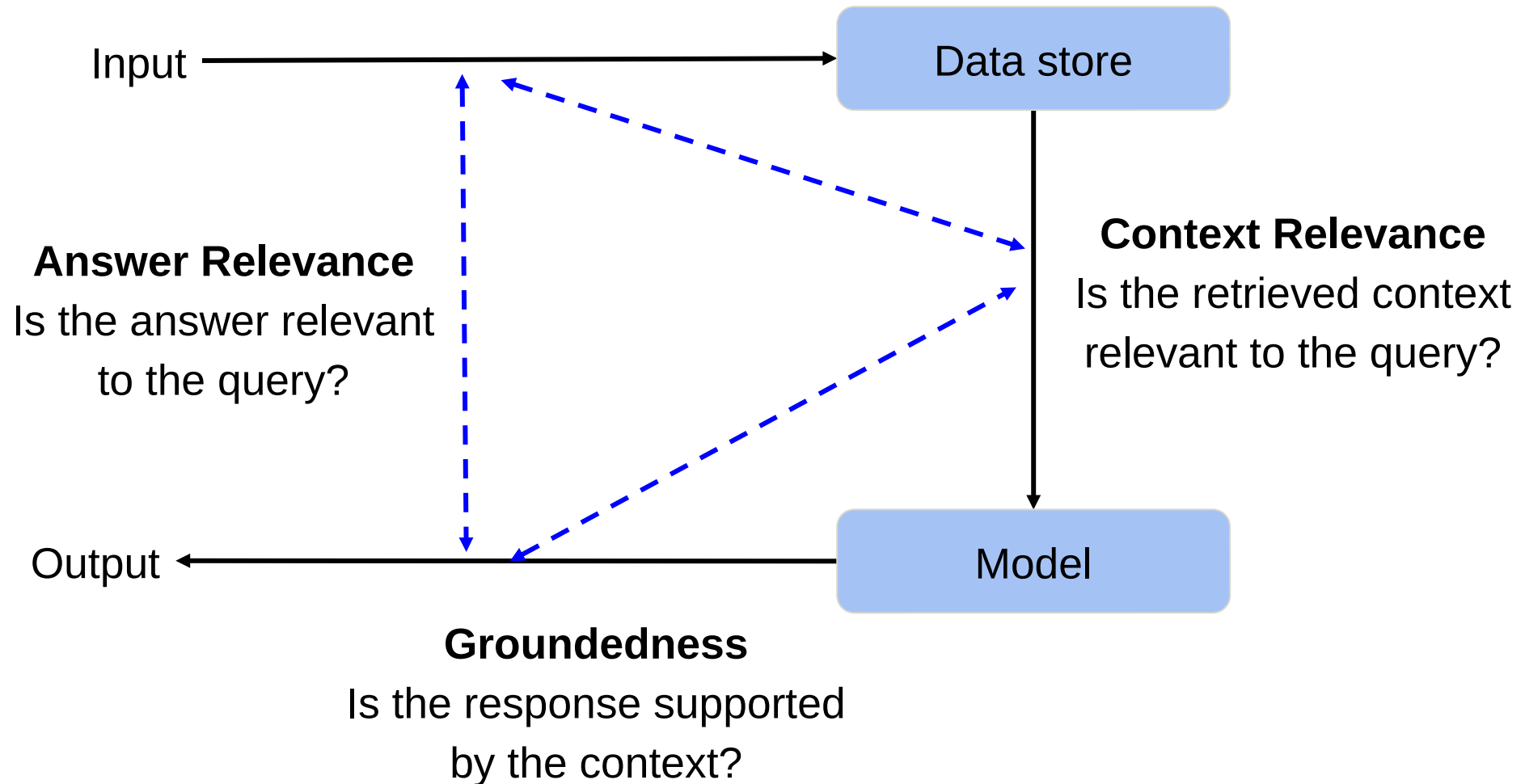
Include instructions to ensure that response is aligned with retrieved results

LLM may be finetuned on RAG prompts

Functional evaluation: The RAG Triad



Functional evaluation: The RAG Triad

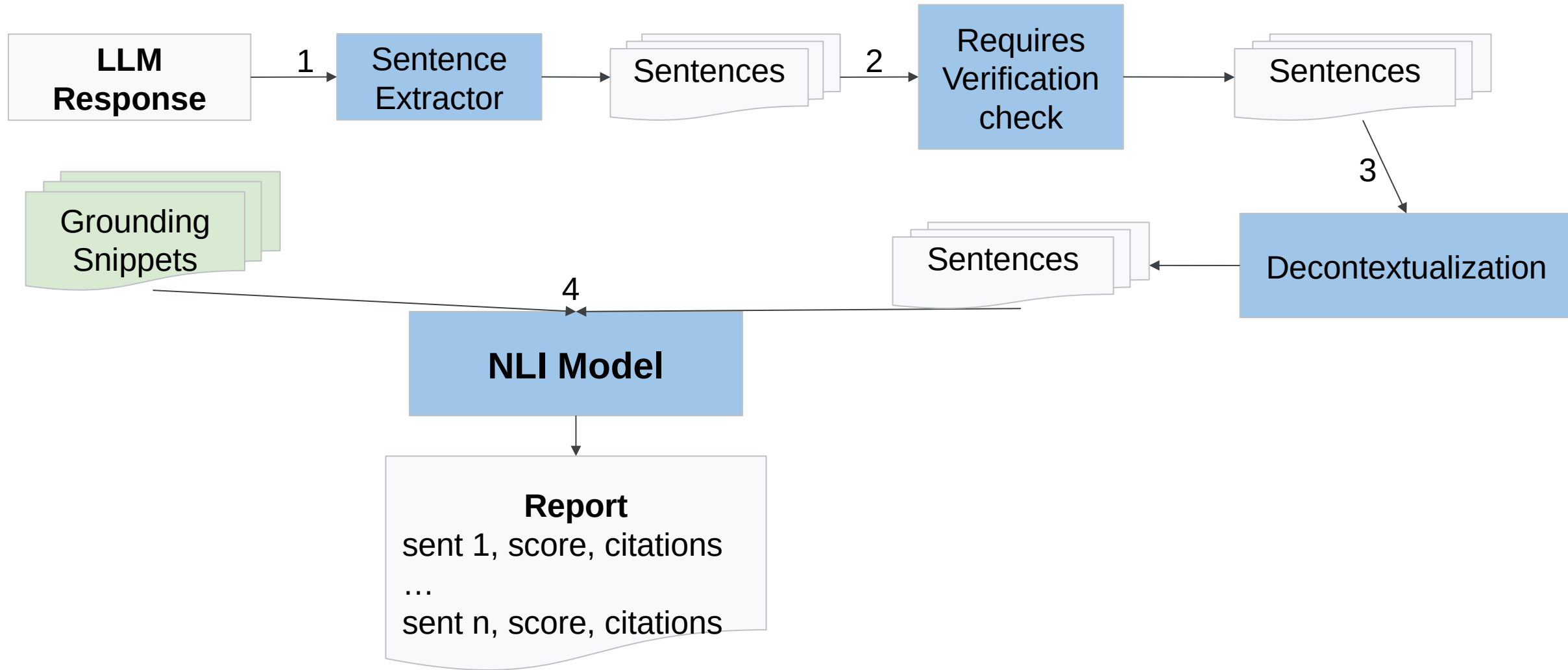


Groundedness

**An LLM response is grounded in a knowledge corpus
if
every claim in the response
can be attributed to
some document in the corpus**

Rashkin, H., Nikolaev, V., Lamm, M., Aroyo, L., Collins, M., Das, D., ... & Reitter, D. (2023). Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4), 777-840.

Corroboration Workflow

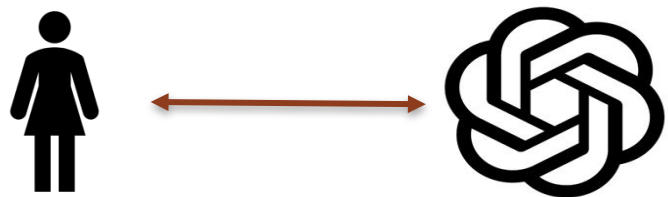


Trust Perspective for RAG architecture

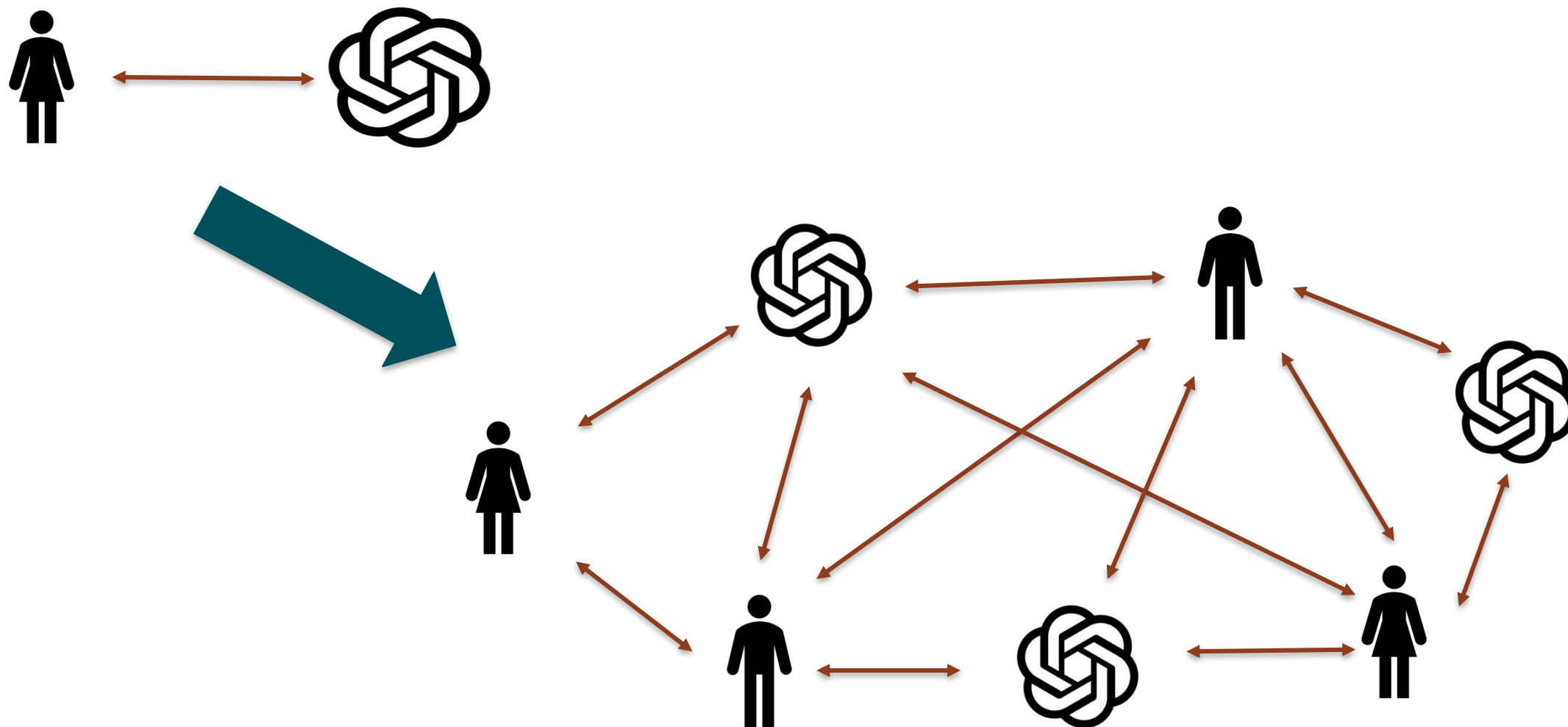
- **Input checking may be inadequate**
 - Jailbreaking: subvert restrictions added by application
 - AI-based input checking has same vulnerabilities as prompting
- **Data and model outputs are returned to user**
 - Can leak information, compromise privacy
 - *Greater* risk than prompting: database may have proprietary information
 - Alignment considerations: bias, cultural sensitivity, ...
- **However RAG triad, emerging tools provide testing, assurance**
 - Structured architecture supports structured engineering practices

Agentic Generative AI

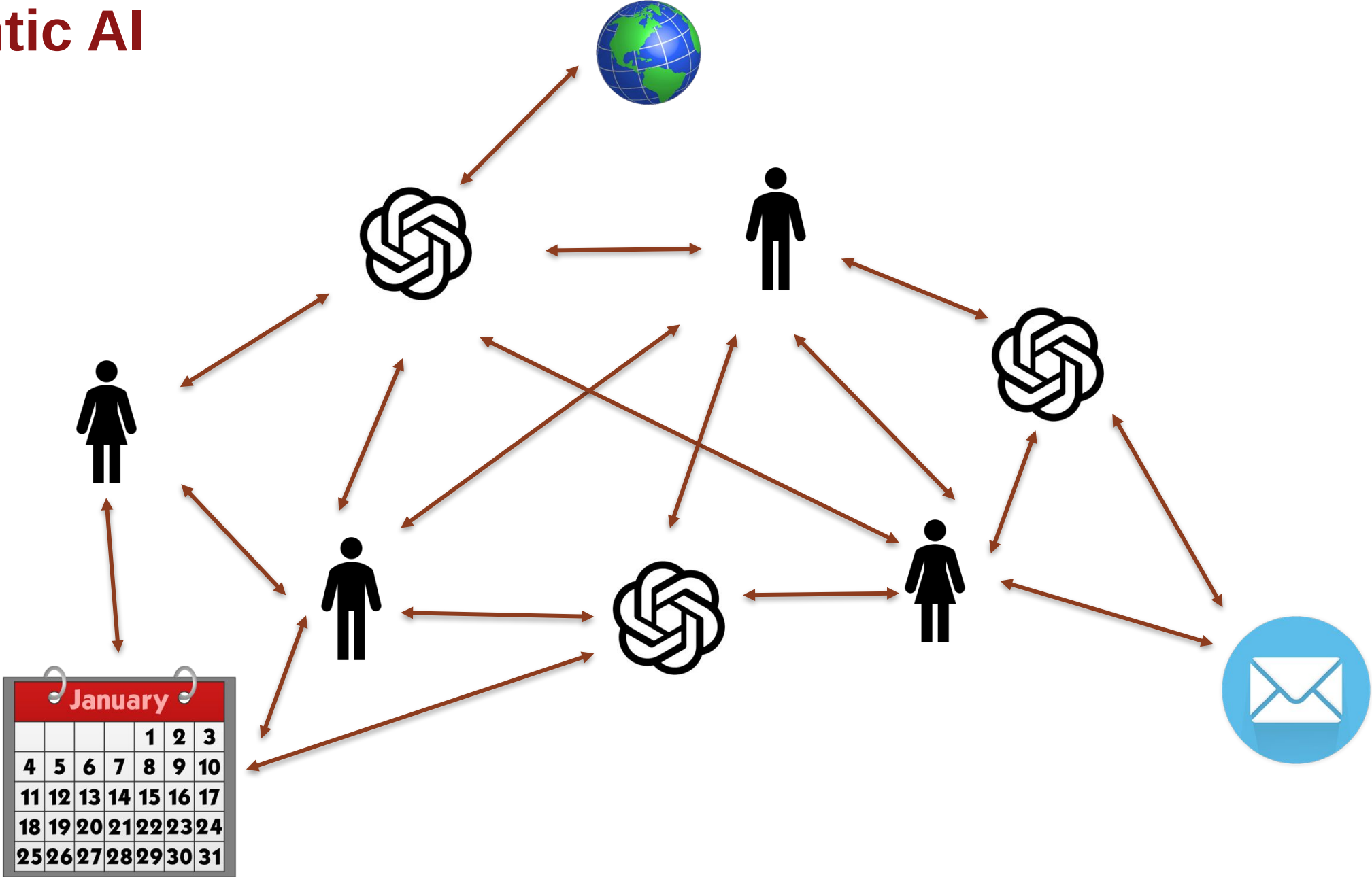
Agentic AI



Agentic AI

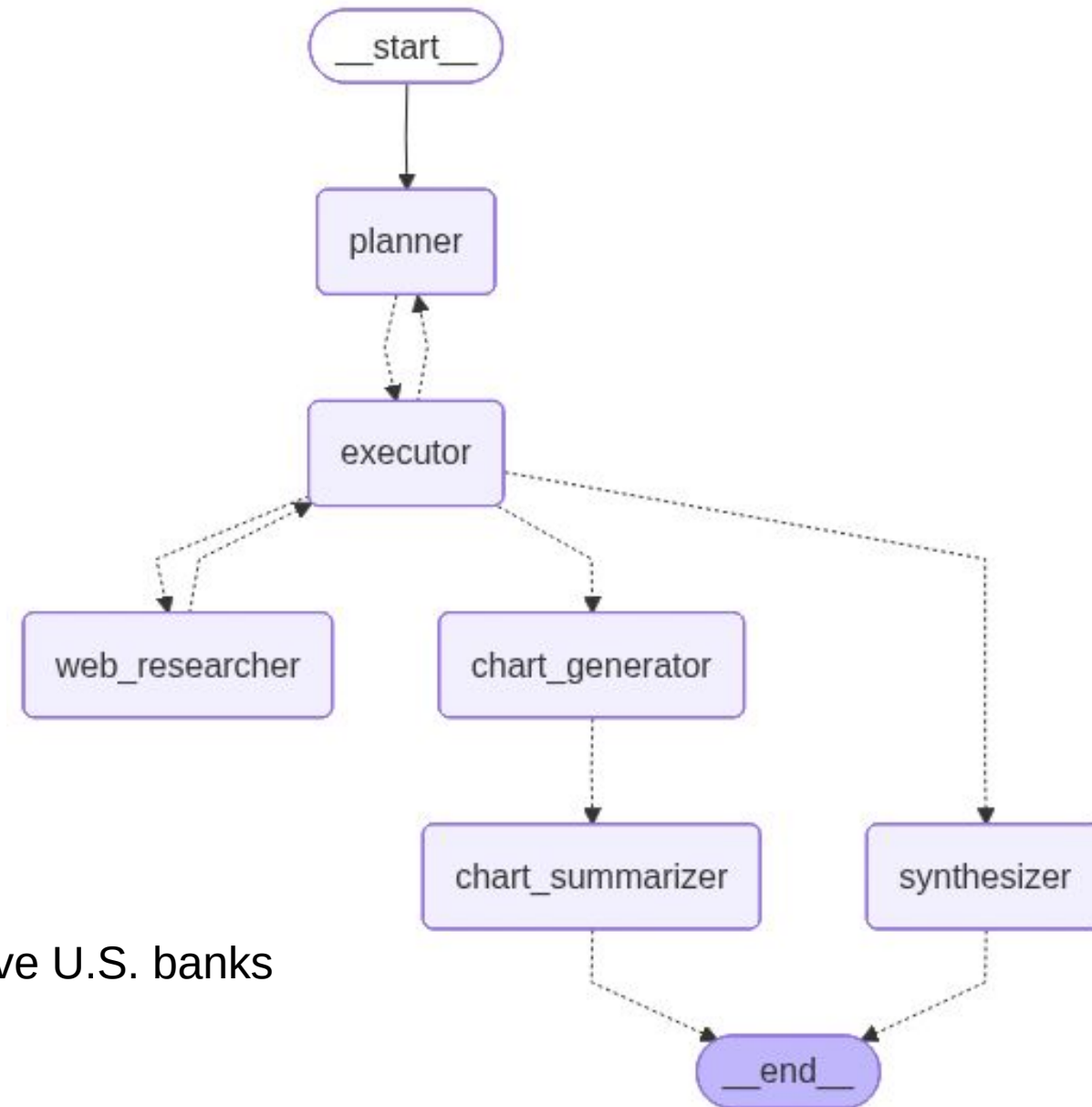


Agentic AI



Agentic Example: Web Research

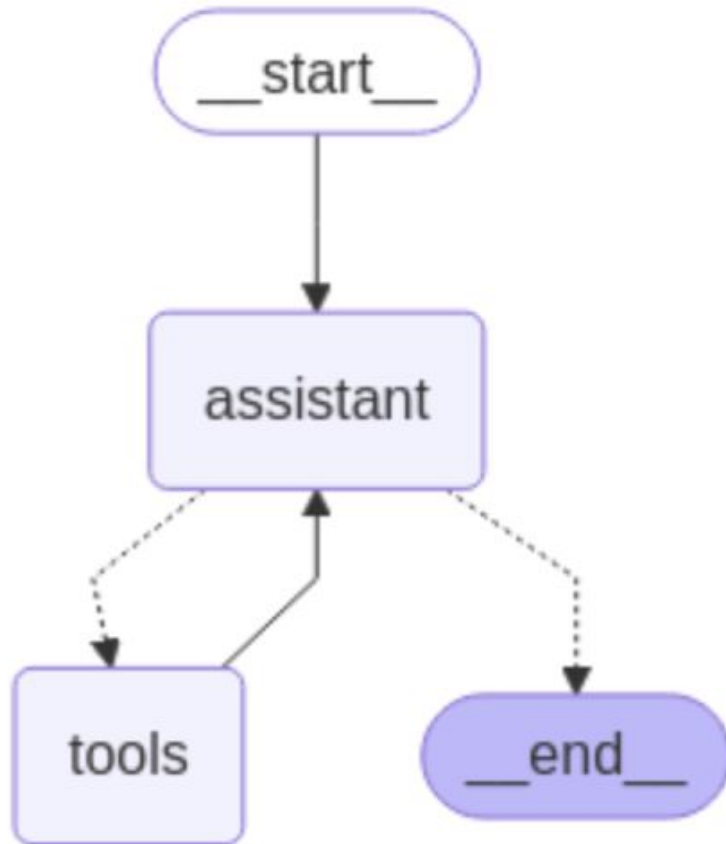
- Memory
 - Augmented LangGraph State
- Reflection
 - Performed by Executor/Planner
- Tool Use
 - Web Researcher uses Tavily Tool
 - Charting uses Python Tool
- Planning
 - Performed by Planner
- Multi agent collaboration
 - Coordinated through Planner and Executor



Query: Chart the market cap (in USD) of the top five U.S. banks

Are multi-agents better than single-agent systems?

Single-Agent Researcher/Chart Generator (o1 model)



“You are an intelligent agent coordinating a multi-step reasoning and tool-use process to fulfill user requests involving data-driven chart generation and trajectory evaluation. You have access to two tools:

- `tavily_tool`: Use to look up exact numerical data from reliable sources - do not infer or estimate.
- `python_repl_tool`: Use to create charts from clean, structured data.

Your goals: Understand the user's request and identify whether:

- More factual information is needed: Use `research_tool`.
- You already have clearly labeled numerical data ready for plotting: Use `chart_generator`.
- The chart is complete and the request has been fully addressed: Proceed to evaluation and explain the reasoning.

Rules for Tool Use:

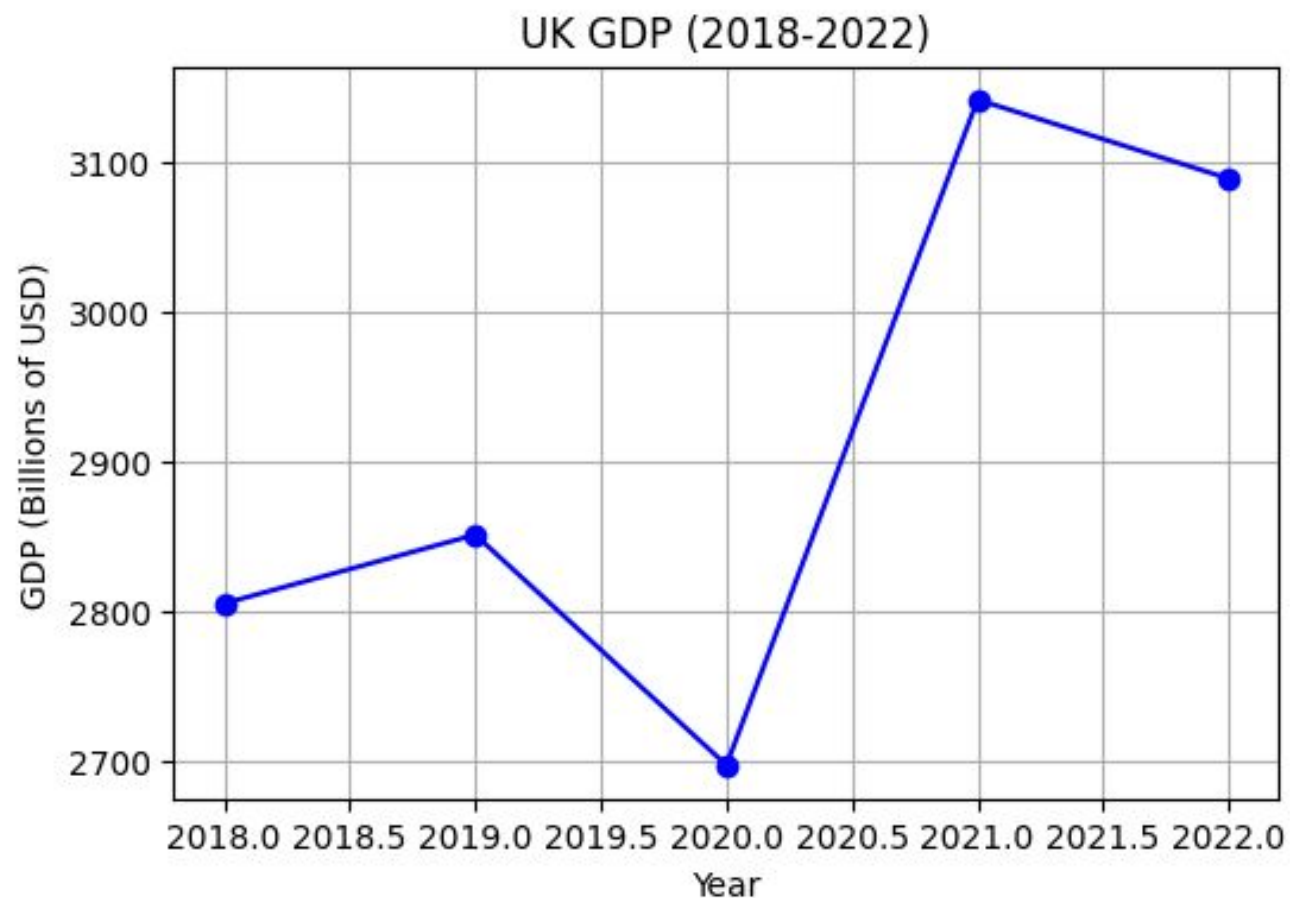
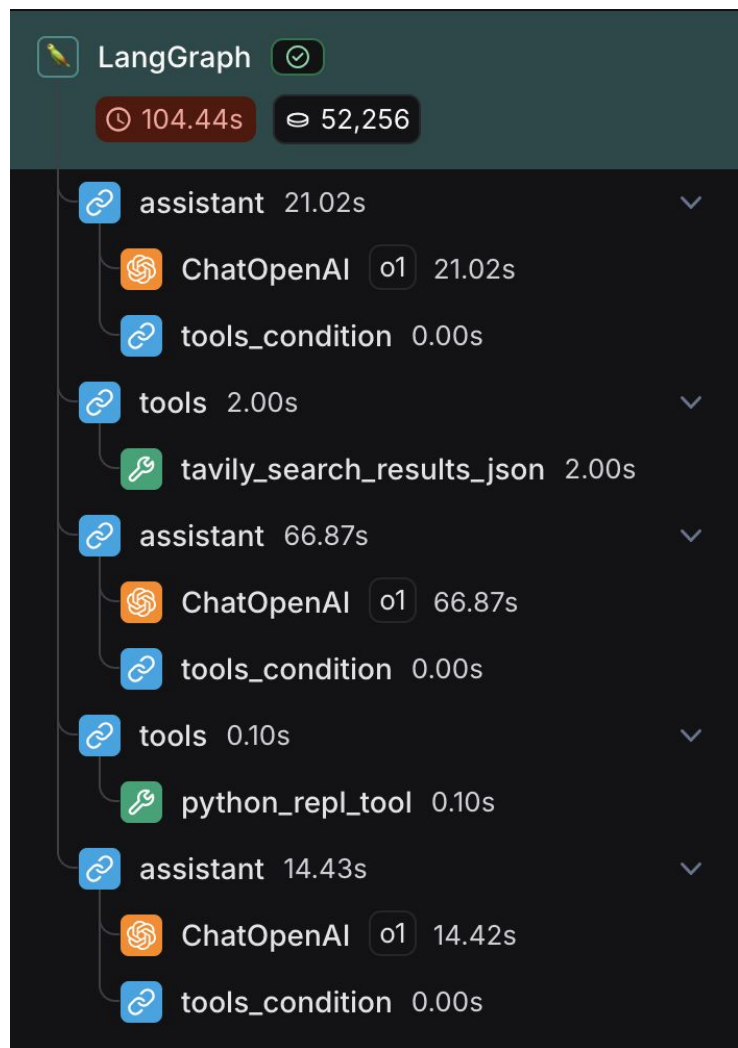
- Use `tavily_tool` if: the user's request lacks exact values or cites only trends. You need more data to complete a chart or confirm a claim.
- Use `chart_generator` if: You or the user already provided a full set of numerical values that can be plotted directly.
- Do not attempt to plot vague, estimated, or incomplete data.

After you have completed the task and before you finish, evaluate your own process. Write a brief assessment structured into four parts:

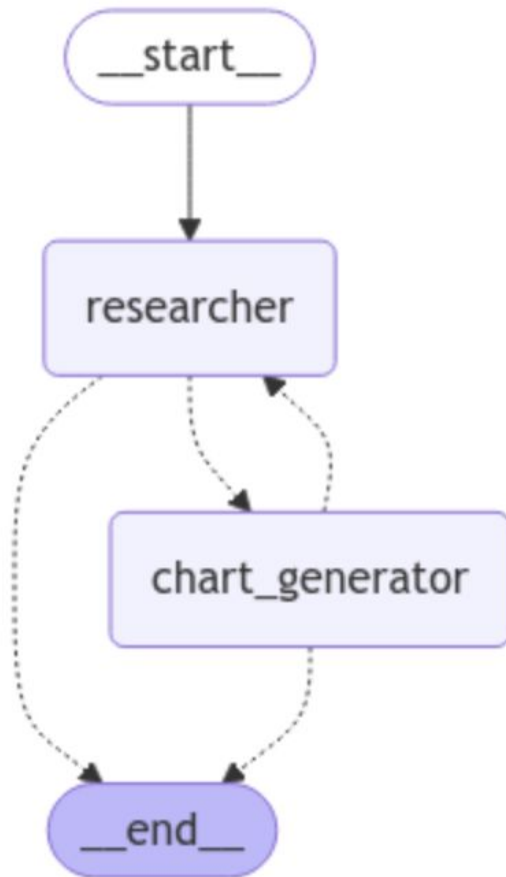
- Summary: Was your overall plan appropriate?
- Strengths: What did you do well (correct tool use, ordering)?
- Weaknesses / Improvements: Any unnecessary steps or logic flaws?
- Recommendations: How could you improve future performance?

Keep responses logical, concise, and focused on moving the task forward.

Single Agent Run



Multi-Agent Chart Generator: Agents Prompts (simplified)



System Prompt (both agents): "You are a helpful AI assistant, collaborating with other assistants. Use the provided tools to progress towards answering the question. If you are unable to fully answer, that's OK, another assistant with different tools will help where you left off. Execute what you can to make progress. Only prefix your response with FINAL ANSWER so the team knows to stop if you or any of the other assistants have successfully generated the chart or responded with the correct deliverable."

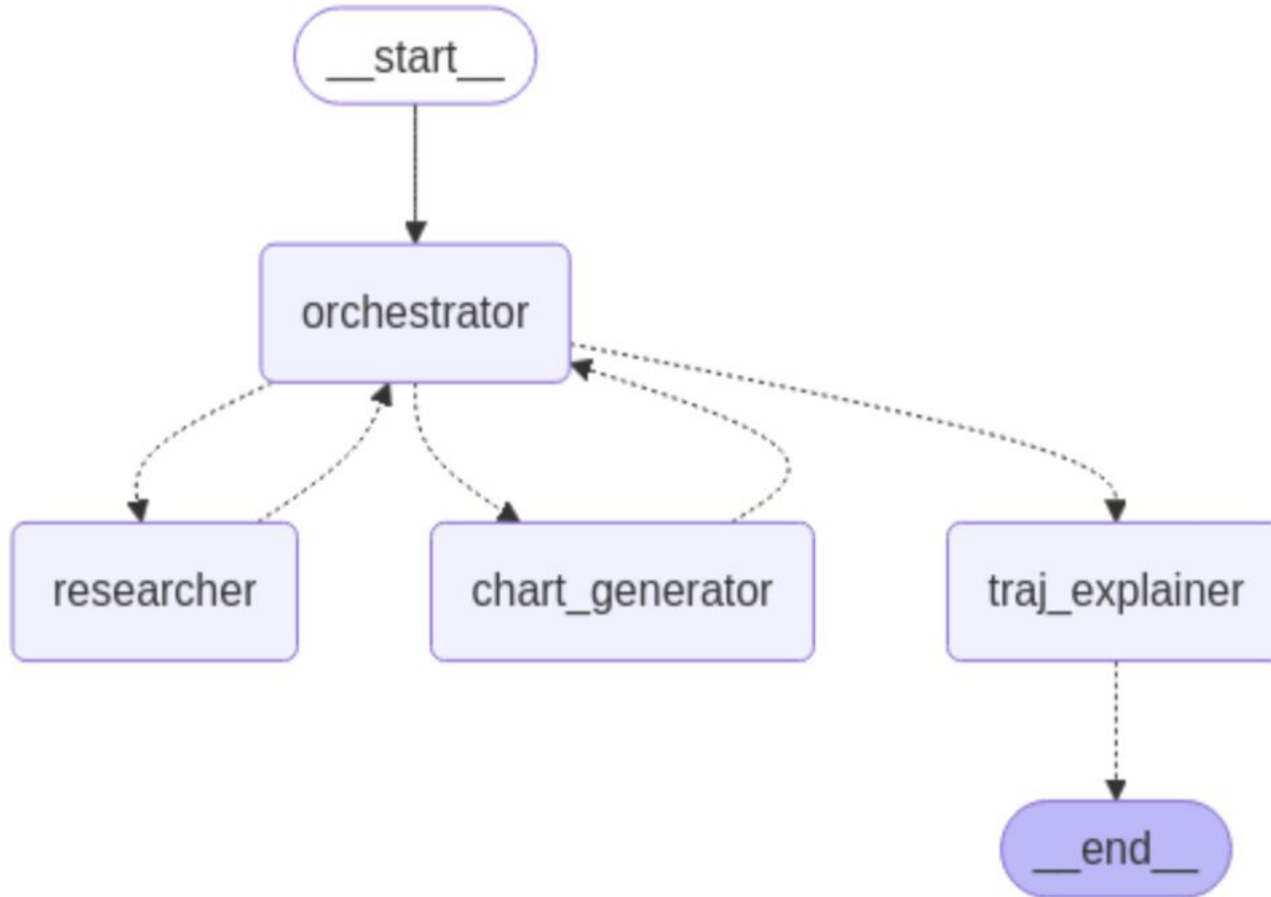
Research Agent: "You can only do research. You are working with a chart generator colleague. After you gather the data you need to plot a chart, ask the chart generator to plot it."

- Bound with Tavily Web Search tool (conduct research)

Chart Generator Agent: "You can only generate charts using Python. You are working with a researcher colleague."

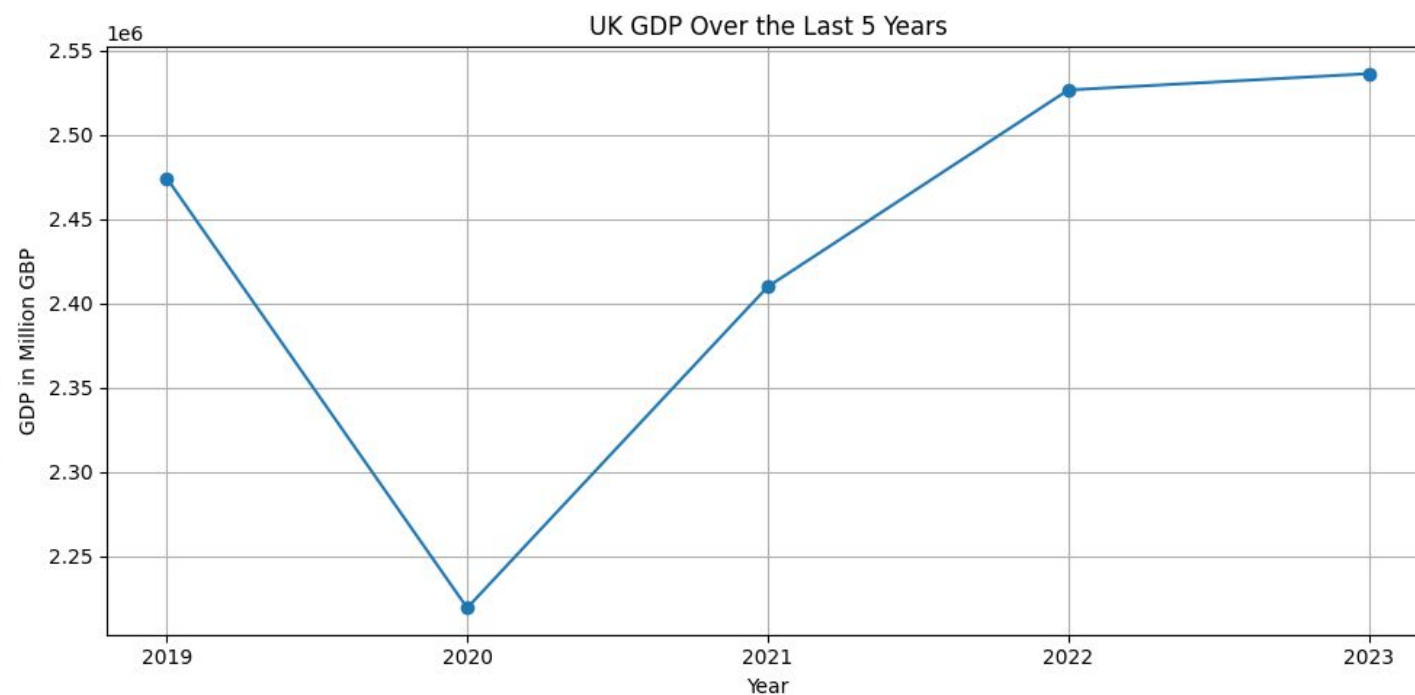
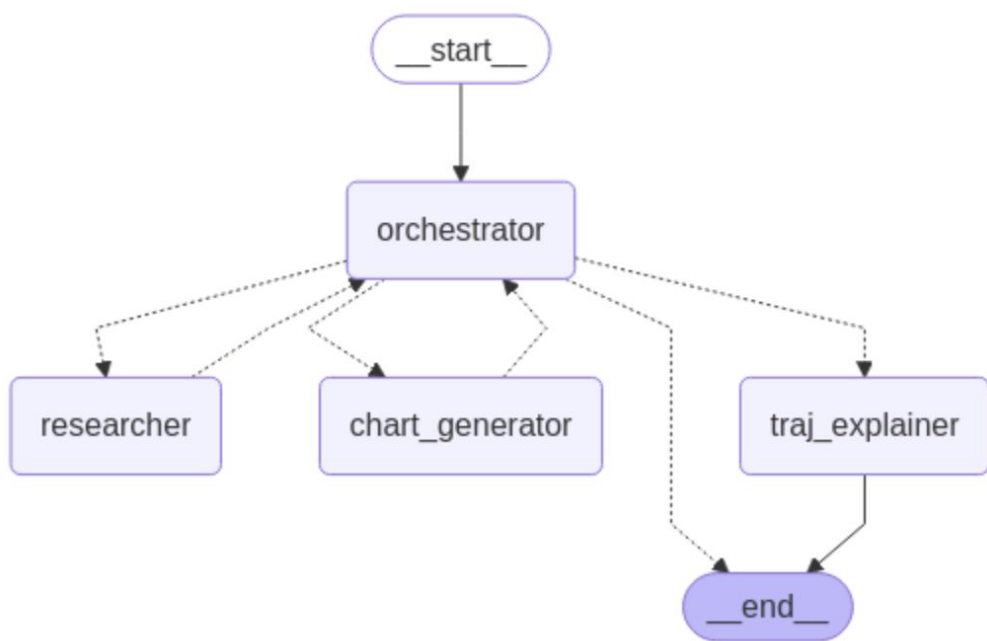
- Bound with Python REPL tool to execute Python code (generate plot)

Improvement: orchestrator and explainer

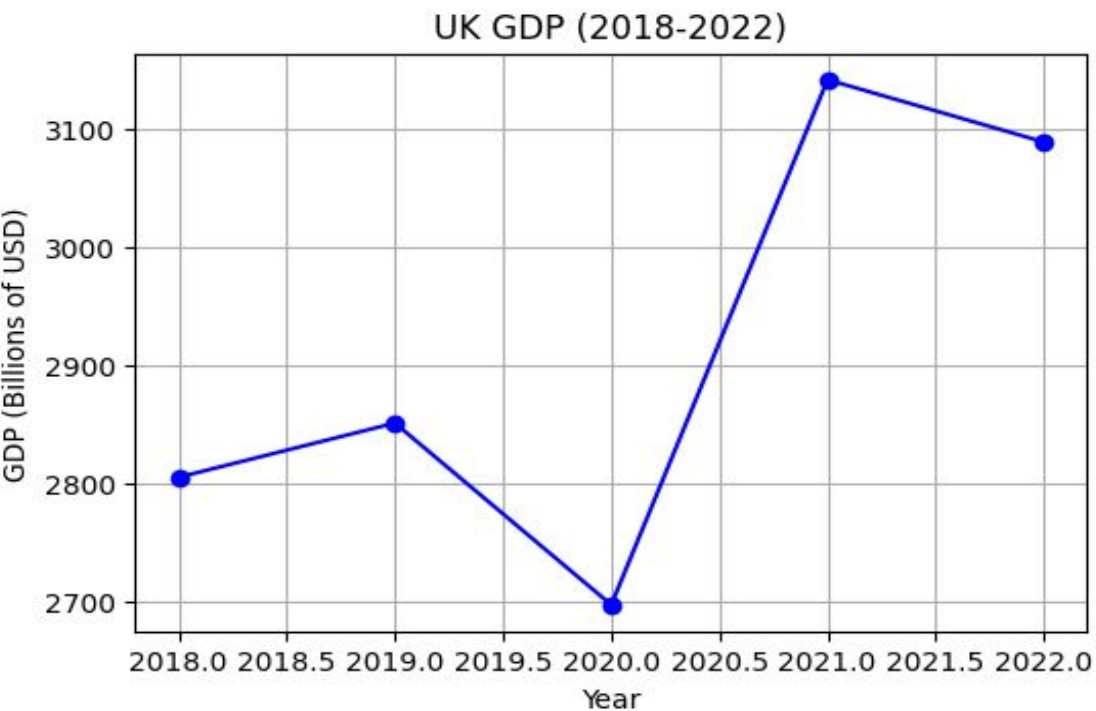


- Orchestrator oversees **researcher** and **chart_generator**
 - Observed improved behavior and traces in chart generation with orchestrator
 - Sometimes, researcher will not provide the numeric values necessary for plotting
- **Goal:** Generate better interaction for orchestrator and researcher
- **Goal:** Failure mode statistics

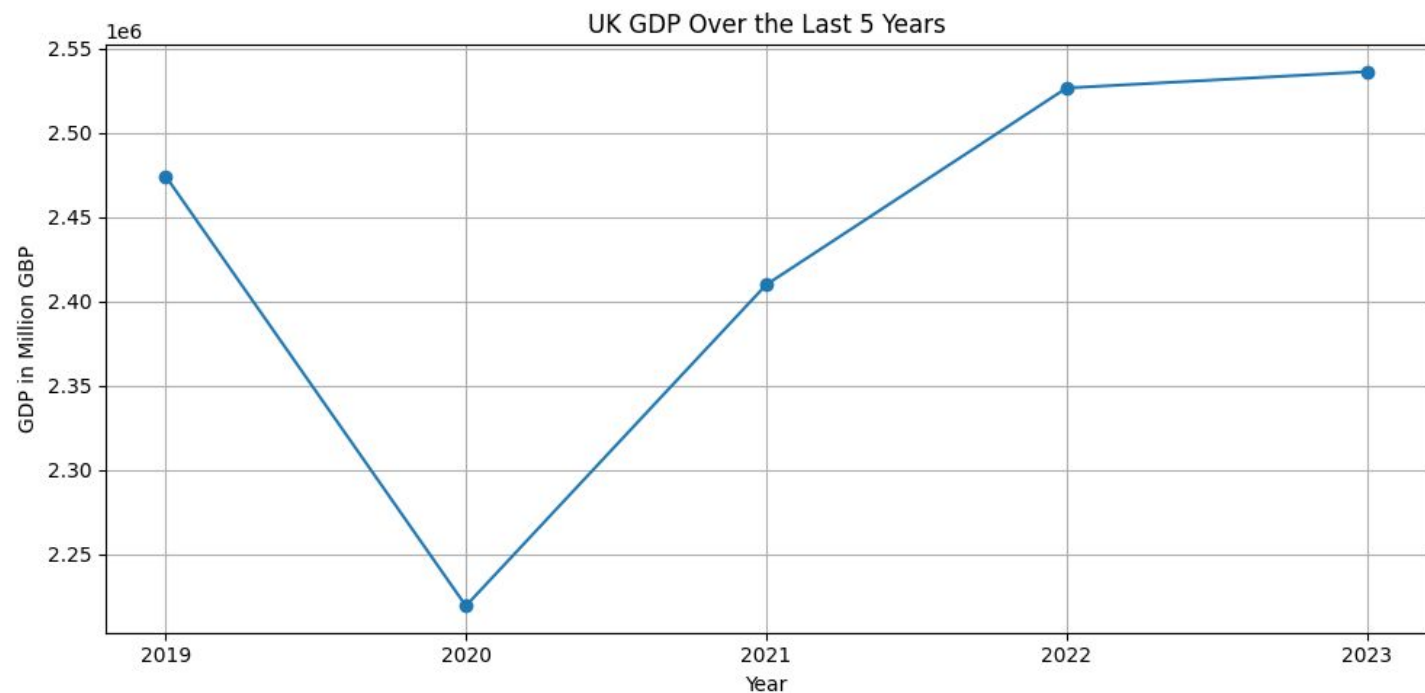
Orchestrator Multi-Agent Run



Comparison



Single agent

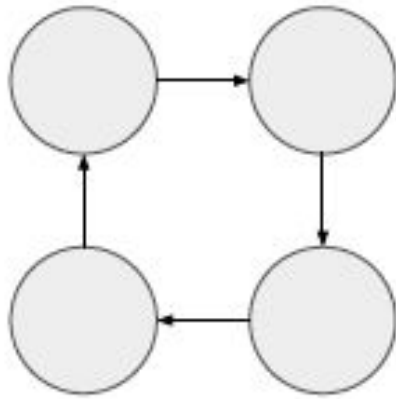


Multi-agent system

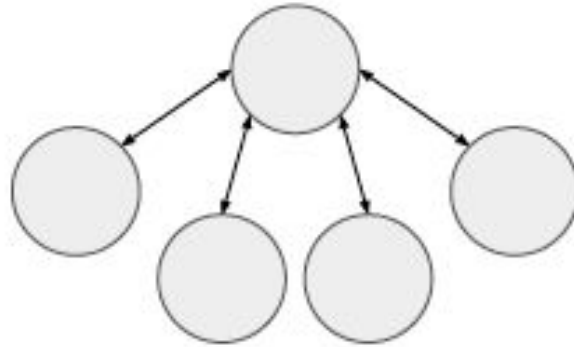
Single Agent vs Multi-Agent Systems

Aspect	Single-Agent Systems	Multi-Agent Systems
Simplicity	Simpler to design, implement, and maintain due to a centralized architecture.	More complex to develop and coordinate, as they involve multiple interacting agents.
Efficiency	Well-defined, narrow tasks with clear objectives. More consistent performance.	Complex, dynamic tasks. Useful for parallel processing.
Scalability	Limited scalability; expanding capabilities often requires significant redesign.	Highly scalable; new agents can be added to handle additional tasks without changing the entire system.
Adaptability	Less adaptable to changing environments; may struggle with unforeseen scenarios.	Highly adaptable; agents can learn and adjust behaviors based on interactions and environmental changes (Galileo)
Fault Tolerance	Single point of failure; if the agent fails, the entire system may be compromised.	Robust against individual agent failures; other agents may be able to compensate, ensuring continued operation.
Communication	Minimal overhead communication, given one agent. Observability issues?	Requires communication protocols for coordination among agents (MCP might help)?
Resource Utilization	Generally lower development and operational costs due to simplicity.	Potentially higher costs due to complexity, but improved performance (pre-reasoning). Can use different models
Use Cases	Ideal for tasks like data entry, simple decision-making, and routine automation.	Suited for complex applications such as autonomous vehicles, distributed sensor networks, and collaborative robotics.

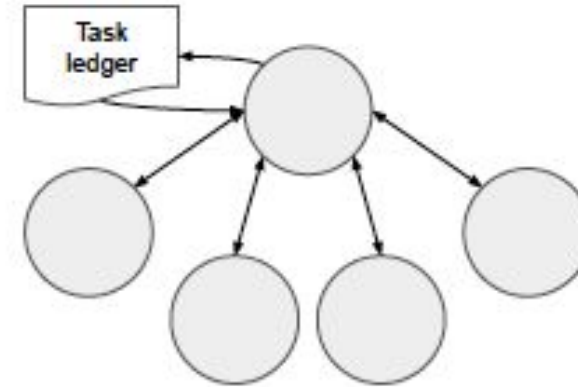
Multi-agent control topologies



(a) Round Robin



(b) Central Orchestrator



(c) Central Orchestrator (Ledger)

- Round robin: Agents are consulted in a set order, determined when the MAS is instantiated. The MAS is a cyclical, deterministic network.
- Central orchestrator: A central orchestrator agent knows about all other agents in the network and determines at each step an appropriate agent to take action. The MAS is a hub-and-spoke, non-deterministic network.
- Central orchestrator with external data structures: More advanced versions of the central orchestrator model, like Magentic-One (Fourney et al., 2024), combine dynamic central orchestration with an external task ledger loop to ensure that the MAS is on-task, increasing overall system effectiveness. This MAS is also a hub-and-spoke, non-deterministic network

More general approach: toward systematic evaluation

Query: “First, get the UK's GDP over the past 5 years, then make a line chart of it”

Ideal Sample Trace:

Planner: “1. Call web_researcher to access last 5 years of UK GDP, 2. Call chart_generator to generate line chart”

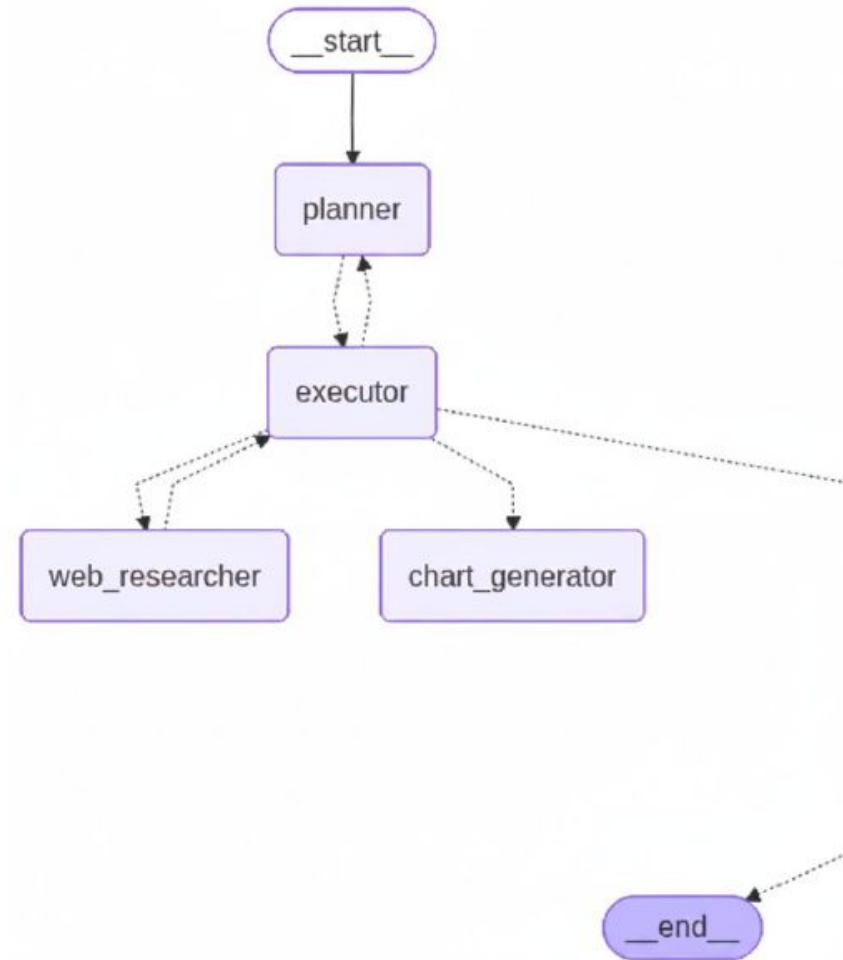
Executor: **route to web_researcher**

Web_Researcher: **calls Tavily web search** “According to [Macrotrends.net](#), the last 5 years of UK GDP is...”

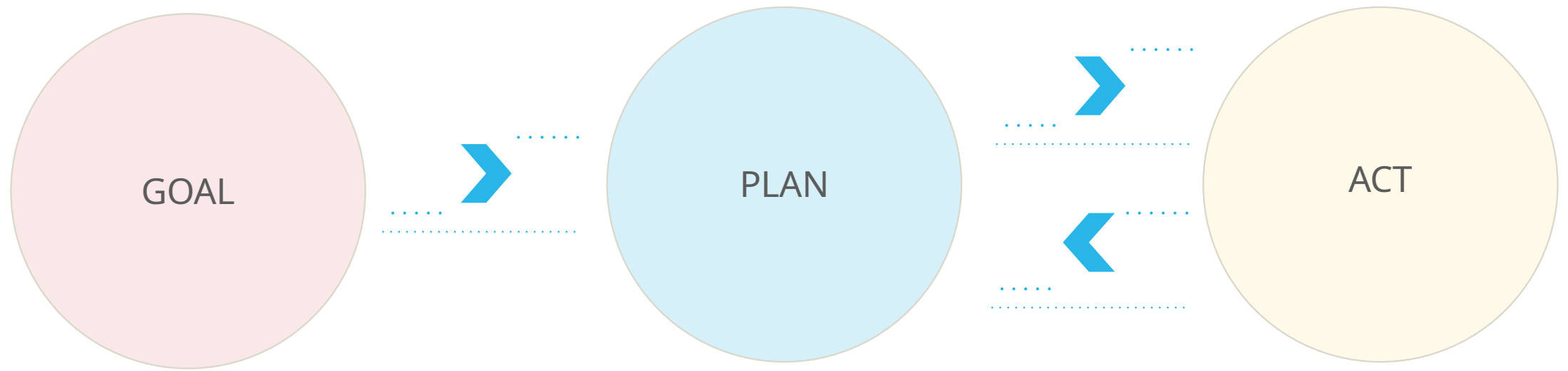
Executor: **route to chart_generator**

Chart_Generator: **calls Python REPL** “Here is a line chart showing the last 5 years of UK GDP...”

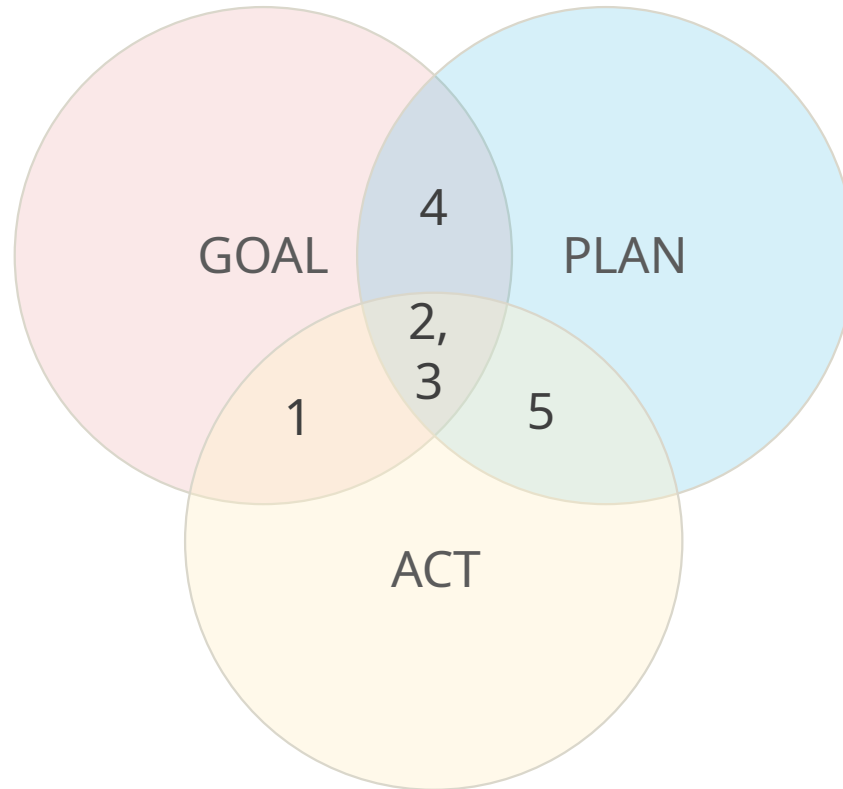
Executor: **routes to _end_**



How Do Agents Work?



What is Your Agent's GPA?



1. Goal Fulfillment

1A. Answer Relevance

2. Logical Consistency

3. Execution Efficiency

4. Plan Quality

4A. Tool Selection

5. Plan Adherence

5A. Tool Calling

GPA: Logical Consistency



LC Failure Mode Example:

Orchestrator: “Given that the current year is 2025, I will call the Researcher agent to access the last 5 years of UK GDP data”

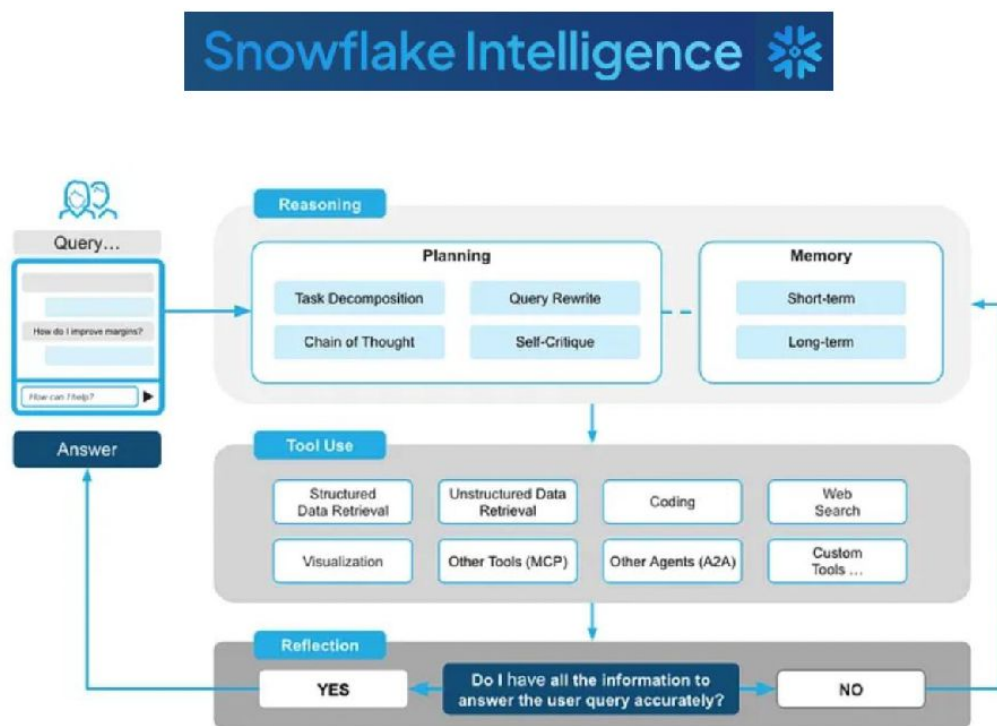
Researcher: ***without calling web_search*** Based on the GPA estimates of other European countries, the last 5 years of UK GDP data is...

Rubric Attributes:

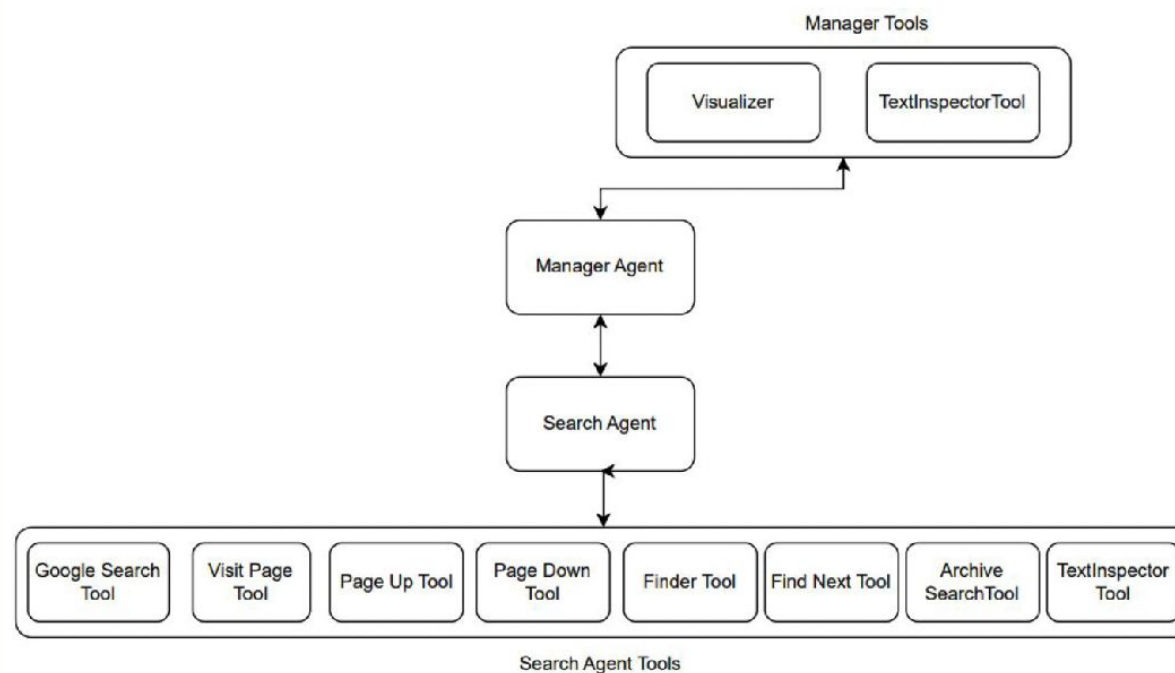
- Every action, claim, and transition in the trace is explicitly justified using information available in the prior context. Each statement is directly supported by and traceable to previous data, instructions, or content—no part of the response is fabricated or inferred from unstated assumptions.
- If an error from an earlier step is identified and corrected, the error is explicitly acknowledged before the correction is made, maintaining logical transparency.
- Each previously assigned task and system instruction is followed.
- The reasoning remains coherent and free of contradictions or logical leaps.

How Can We Trust These LLM-Judges?

LLM-Judge Benchmarking!



TRAIL

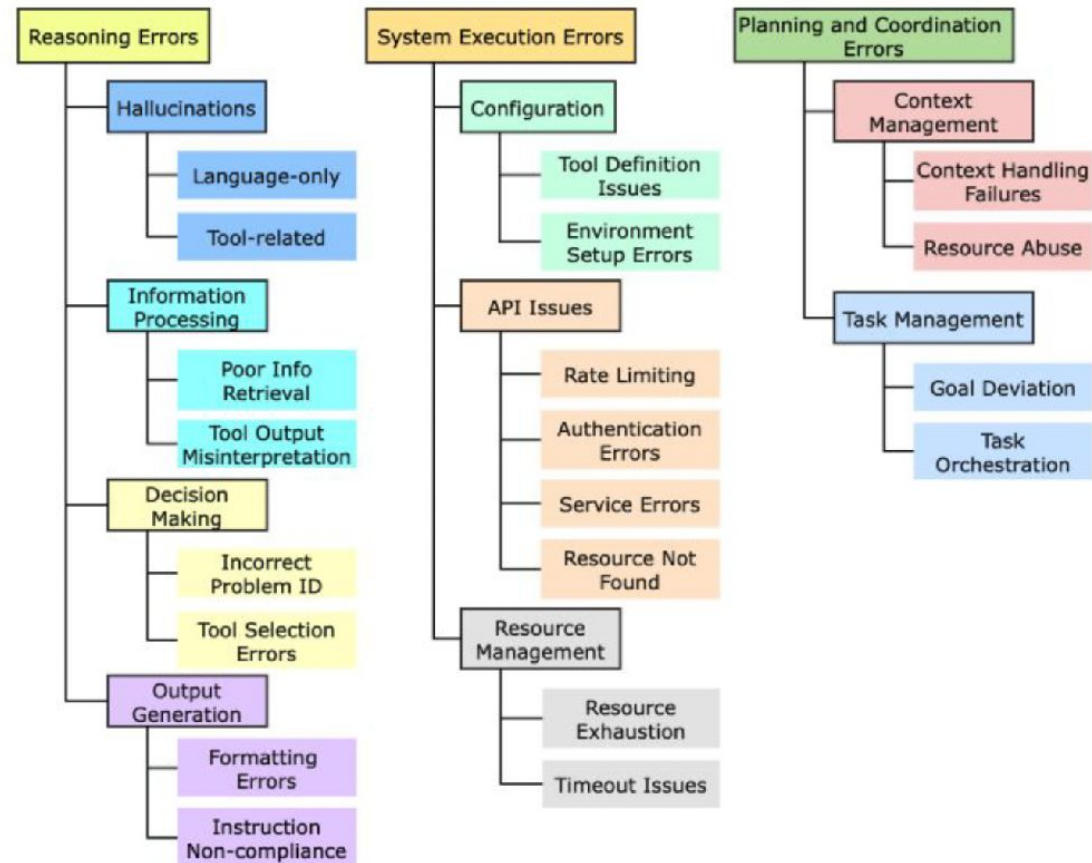


What is TRAIL¹?

148 human-annotated traces generated on GAIA and SWE-Bench

- **GAIA (117): Test agent's general reasoning ability with multi-modality, web-browsing, web-navigation**
- SWE-Bench (31): Tests agent coding ability to generate code patch given GitHub codebase and description of issue

Example [agent trace](#) & [annotation](#) walk-through



¹ Deshpande, Darshan, et al. "TRAIL: Trace Reasoning and Agentic Issue Localization." *arXiv preprint arXiv:2505.08638* (2025).

TRAIL Benchmarking: Methodology

1. Pre-process all TRAIL/GAIA spans into readable format
 - a. Readable Trace Walkthrough
2. Human Review: Map TRAIL errors to GPA dimensions
3. GPA LLM-Judges, TRAIL LLM-Judge Initialization
 - a. *Claude-4-sonnet* base model
 - b. Custom Instruction: High level description of agent system
 - c. Custom Instruction: 1-2 few shot examples for each metric (dev set)
 - d. Custom Instruction: Output template
4. LLM-Judge Error Coverage
5. LLM-Judge Alignment with Human Judgment
6. LLM-Judge Localization

Results

- **The Agent GPA framework systematically identifies a broad range of agent failures**
 - In particular, it captures all the 281 agent internal errors on our test set split of the TRAIL/GAIA dataset
- **LLM Judges measuring Agent GPA show strong agreement with human judgments**
 - TRAIL/GAIA dataset:
 - 94% (264/281) agreement
 - Baseline: TRAIL LLM Judge shows 55% (154/281) agreement
 - Snowflake Intelligence:
 - 82% agreement when grading on a 3-point scale
 - 97% agreement when grading on a 2-point scale
- **LLM Judges measuring Agent GPA localize errors in strong agreement with human annotations**
 - TRAIL/GAIA dataset:
 - 85% (240/281) agreement
 - Baseline: TRAIL LLM Judge shows 49% (138/281) agreement

GPA LLM-Judge Alignment with Humans

Motivation: To prove that GPA LLM-Judges have strong alignment with human judgement, *each* GPA LLM-Judges must 1. catch the same errors as human annotations and **2. (numerically) score each trace's GPA dimensions similarly to humans.**

Human Score Alignment: Does each LLM-judge score the GPA dimension of each trace similarly to a human?

Table 4: GPA Judge Alignment with Human Judgment

Judge	Dev			Test		
	Acc-OB1	Acc-3pt	Correl	Acc-OB1	Acc-3pt	Correl
LC	0.983	0.793	0.626	0.983	0.881	0.764
EE	0.862	0.483	0.513	0.949	0.356	0.623
PA	1.000	0.862	0.869	0.983	0.864	0.917
PQ	0.879	0.690	0.565	0.966	0.695	0.672
TS	0.895	0.719	0.663	0.962	0.868	0.895
TC	0.889	0.667	0.589	0.941	0.725	0.706

(Acc-OB1 = Off-by-one Accuracy, Acc-3pt = Bucketed Accuracy, Correl = Correlation)

[Submitted on 9 Oct 2025 (v1), last revised 27 Mar 2026 (this version, v2)]

What Is Your Agent's GPA? A Framework for Evaluating Agent Goal-Plan-Action Alignment

Allison Sihan Jia, Daniel Huang, Nikhil Vytla, Seung Won Wilson Yoo, Nirvika Choudhury, Shayak Sen, John C. Mitchell, Anupam Datta

We introduce the Agent GPA (Goal-Plan-Action) framework, driven by the fundamental insight that critical agent failures emerge at the intersections of setting goals, devising plans, and executing actions. We operationalize the framework with a factorized suite of LLM judges designed to measure distinct elements of Goal-Plan-Act alignment. To make this methodology scalable and generalizable across diverse agent architectures and datasets, we use state-of-the-art automated prompt optimization techniques to systematically generate domain-specific evaluation criteria. We validate this approach across three benchmarks: a multi-agent research setting (TRAIL/GAIA), a single coding agent setting (TRAIL/SWE-bench), and a private, enterprise data-agent setting (Snowflake Intelligence). Extensive evaluation on TRAIL/GAIA demonstrates the core validity of the framework, which identifies a broad range of agent failures (95% of human-annotated errors), localizes errors to enable targeted debugging (86% of human-annotated errors), and exhibits strong agreement with human evaluators. Crucially, by applying our automated methodology to both public datasets, we demonstrate that our GPA judges generally achieve the highest error coverage (ranging from 76% to 86%) in comparison to manual prompting approaches. We also leverage an evolutionary coding agent to improve judge consistency by up to 38% through iterative refinement of evaluation rubrics. Overall, Agent GPA provides a rigorous and generalizable paradigm for targeted agent evaluation.

Link: <https://arxiv.org/abs/2510.08847>

Security Perspective for Agentic AI

- **Untrusted user input**
 - Can be passed to applications accessible to agent
 - Possible data poisoning
- **Agent actions**
 - Trick one agent into creatively corrupting another
 - Potential serious security breaches using tools accessible to agents (databases, web search,...)
- **Agent output returned to user**
 - Data breaches and information leakage

Prompt optimization



GEPA: System Optimization through Reflective Text Evolution

Optimize text components—AI prompts, code, or instructions—of any system using reflective text evolution.

pypi v0.0.18 downloads 2M DSPy 7059 members

Overview

GEPA (Genetic-Pareto) is a framework for **optimizing arbitrary systems composed of text components**—like AI prompts, code snippets, or textual specs—against any evaluation metric. It employs LLMs to reflect on system behavior, using feedback from execution and evaluation traces to drive targeted improvements. Through iterative mutation, reflection, and Pareto-aware candidate selection, GEPA evolves robust, high-performing variants with minimal evaluations, co-evolving multiple components in modular systems for domain-specific gains.

This repository provides the official implementation of the GEPA algorithm as proposed in the paper titled "GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning" (<https://arxiv.org/abs/2507.19457>). In order to reproduce experiments from the paper, we provide a separate [reproduction artifact](#).

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, Omar Khattab

Leverage train-time textual feedback to perturb prompts

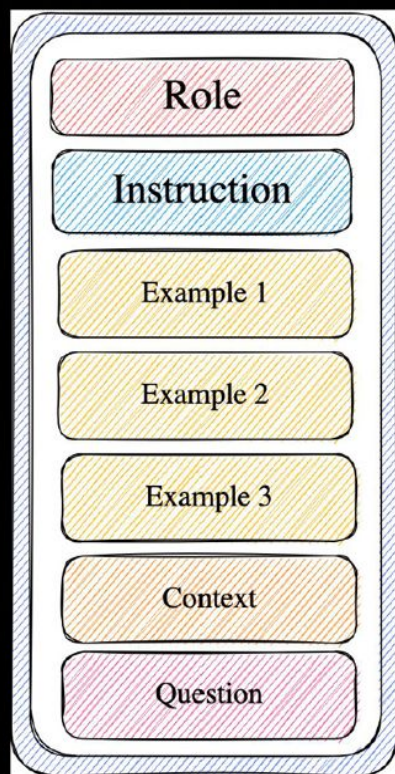
Genetic

Discovers test case specific solutions first and aggregate later

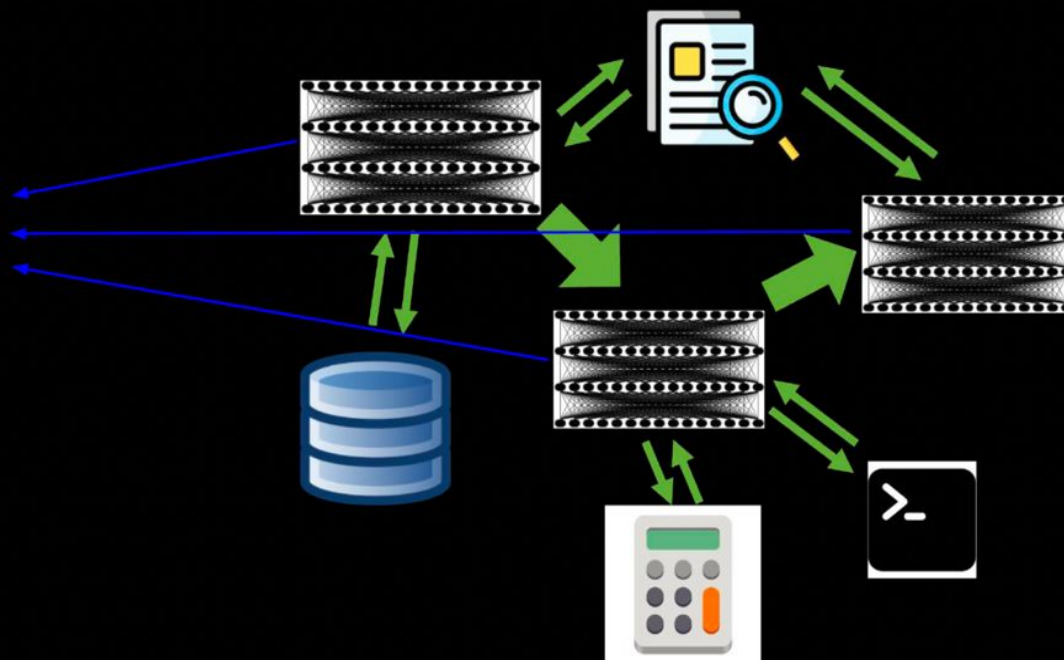
Pareto (Multi Objective)

Credit: slides from talk by
Lakshya Agrawal

How to optimize a Compound AI System for complex tasks in domains facing **sample and data efficiency challenge**



Prompt



Compound AI System

Tool
Invocation

Retrieval

Guardrails

Insight

It is possible to use *very few* data and rollout samples, to reflectively update the **prompt** making a large and successful change to the model, as compared to weight updates, which requires much more data



Problem

Prior learning techniques only consider numeric scores as learning signal, observed at the end of long-running rollouts

GEPA: Reflection

Use domain-specific textual feedback as learning signal in addition to numeric rewards, making large updates from one rollout!

Problem

Prior prompt optimizers proposed all new prompts upfront, to perform bayesian search over them

GEPA: Genetic

Assume that good prompts are derived from previous good prompts - building a tree of prompts that improve upon each other

Problem

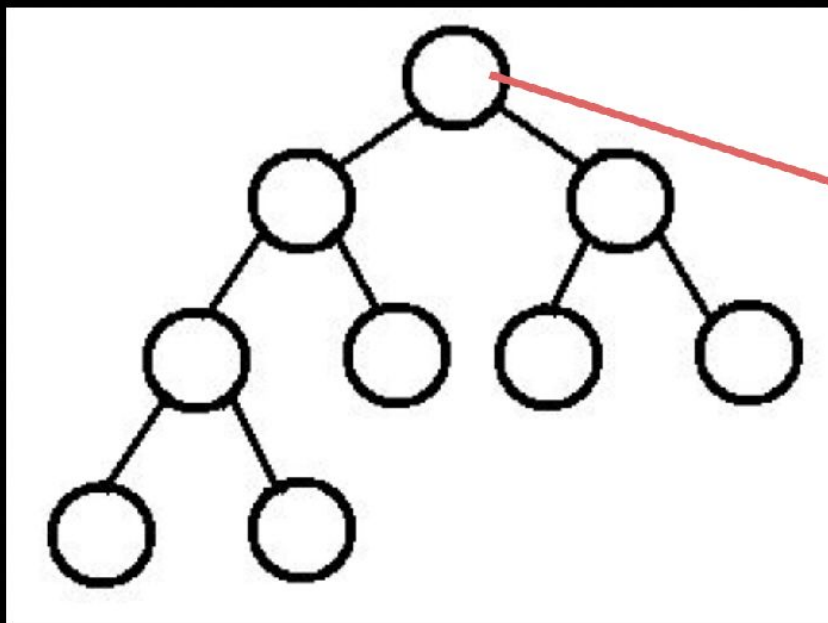
Most AI optimization techniques focus on the **aggregate score** across the full dataset, missing on data point specific improvements

GEPA: Pareto

Focus on improving on individual data points to discover testcase specific solutions, and aggregate learnings later

GEPA for Agent Architecture Discovery

GEPA Search Tree



Each node in the tree represents a **set of prompts** when GEPA is used for **Prompt Learning**

GEPA is a text evolution engine

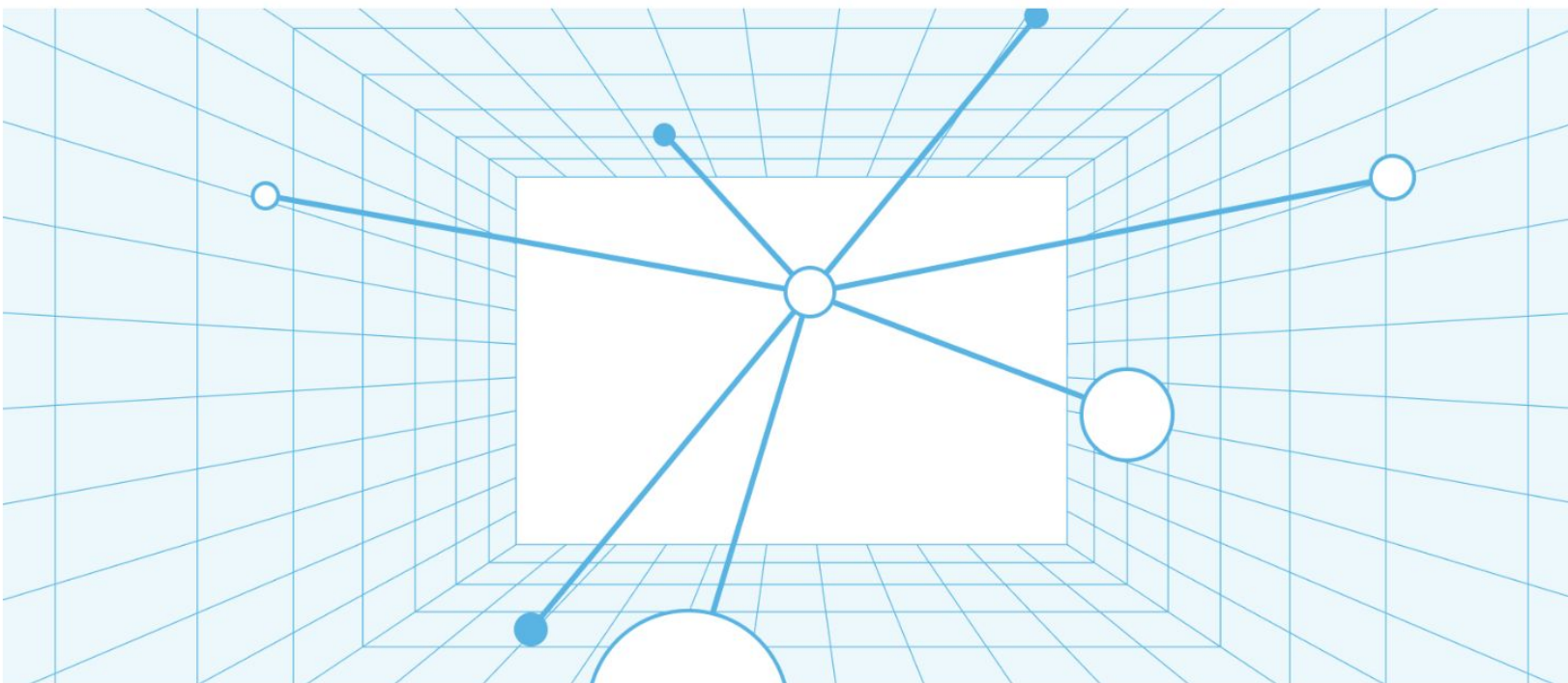
Given a target metric, GEPA will reflectively evolve the text!



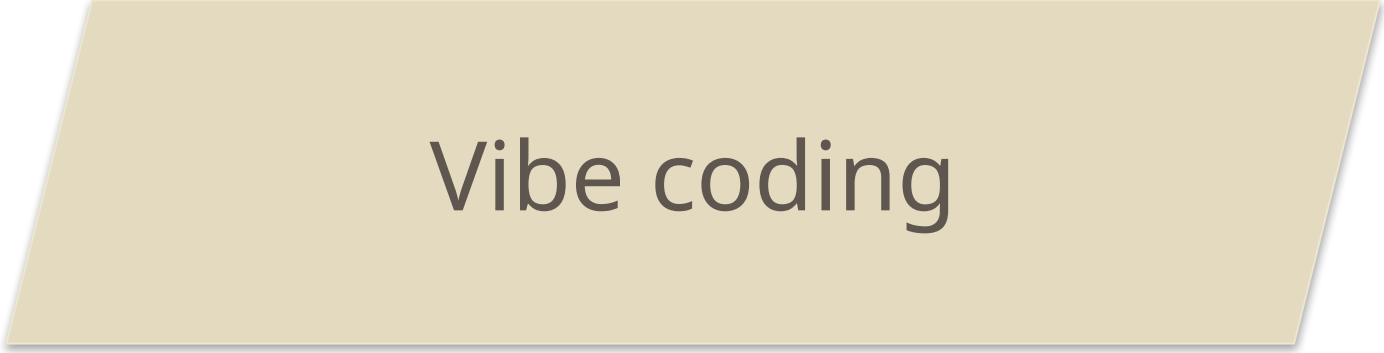
CoCoEvolve: What If a Coding Agent Could Optimize Your AI Systems?



Wilson Yoo +3

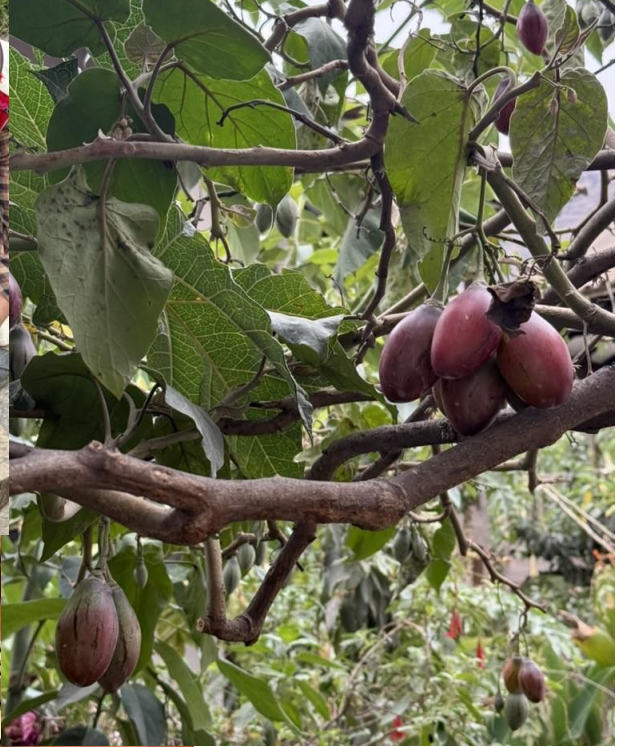


<https://www.snowflake.com/en/blog/engineering/optimize-snowflake-ai-systems-cocoevolve/>

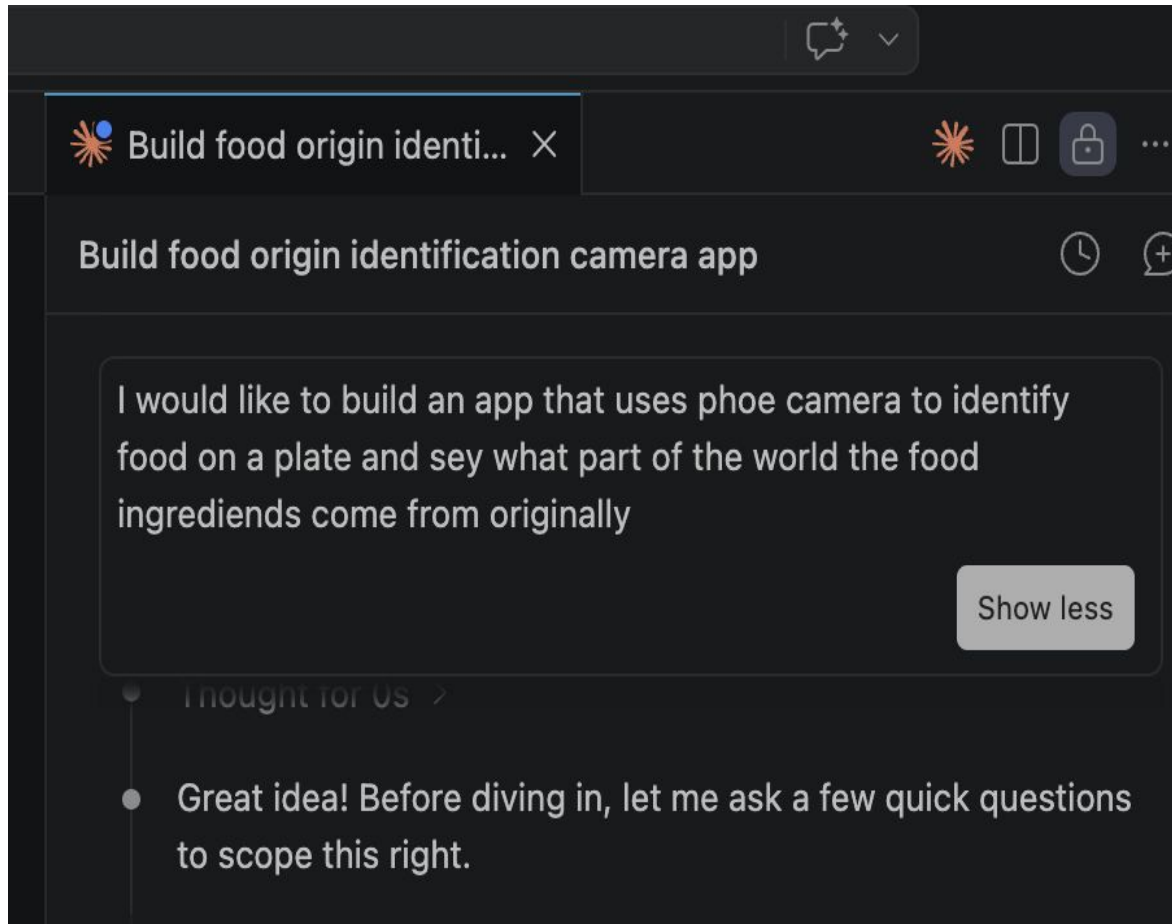


Vibe coding

Building with agentic AI



Describe the app I want to build



What platform do you want to build for?

- ☐ **React Native (iOS + Android)**
Cross-platform mobile app using JavaScript/TypeScript. Ships to both platforms from one codebase.
- ☐ **iOS only (Swift)**
Native iOS app. Best camera access and performance, but Apple devices only.
- ☐ **Web app (mobile browser)**
Works in any mobile browser without install. Uses browser camera API. Easiest to deploy.
- ☐ **Expo (React Native, easier setup)**
Managed React Native — fastest way to get a working prototype with camera access on iOS and Android.
- ☐ **Other**

Answer questions from Claude Code

Platform

AI backend

Output style

×

How do you want to identify the food and trace its origins?

☐

Claude API (vision + reasoning)
Send the photo to Claude. It identifies the food, lists ingredients, and explains where each originates. No separate food database needed.

☐

Claude API + a food/nutrition API
Claude identifies the dish, then a structured API (e.g. Spoonacular) provides ingredient lists, and Claude explains origins.

☐

Not sure — recommend something
Claude will suggest the best approach for your goals.

☐

Other

Platform

AI backend

Output style

×

What should the output look like?

☒

Map + ingredient list
A world map with pins showing where each ingredient originates, plus a list below.

☐

Text breakdown
A clean text result: dish name, ingredients, and a short paragraph or bullets on geographic origins.

☐

Cards per ingredient
Each ingredient gets its own card showing its name, origin region, and a fun fact.

☐

Other

1

Submit answers

Esc to cancel

Claude Code recommends a plan

- Here's my recommendation and plan for you:

Backend: Claude API (vision) — Claude can identify the dish, list all ingredients, and return their geographic origins in one API call. You get structured JSON back with coordinates you can drop directly onto a MapKit map. No extra food database needed, and it handles edge cases (mixed dishes, fusion cuisine) naturally.

App architecture:

- **SwiftUI** — modern, clean UI
- **PhotosUI / AVFoundation** — camera capture + photo library
- **URLSession** — REST calls to Claude API (no official Swift SDK, but the REST API is simple)
- **MapKit** — world map with pins per ingredient origin
- **Claude API** — vision model identifies dish → returns ingredients + origin coordinates + fun facts

Flow:

1. User taps "Scan" → takes photo or picks from library
2. Image is base64-encoded and sent to Claude with a structured prompt
3. Claude returns JSON: `[{ ingredient, origin_country, lat, lng, fun_fact }]`
4. App renders a MapKit map with pins + scrollable ingredient list below

Want me to build this out? I'll create a complete Xcode-ready Swift project. You'll need:

- An [Anthropic API key](#)
- Xcode 15+
- An Apple developer account (to run on a real device with camera)

Claude Code builds the app and tells how to run it

- All 6 Swift files are ready at `~/Desktop/FoodOrigins/`. Here's a summary of what was built:

Files created:

File	Purpose
FoodOriginsApp.swift	App entry point
Models.swift	<code>Ingredient</code> + <code>AnalysisResult</code> data models
ClaudeService.swift	REST calls to Claude API with vision
ContentView.swift	Home screen — camera, photo picker, analyze button
CameraView.swift	Native camera wrapper
ResultView.swift	MapKit world map + ingredient list

How to get it running — short version:

1. Open Xcode → New Project → iOS App → name it `FoodOrigins` (SwiftUI, Swift)
2. Delete the generated `ContentView.swift`, drag in all 6 `.swift` files from `~/Desktop/FoodOrigins/`
3. In the target's **Info** tab, add `NSCameraUsageDescription` and `NSPhotoLibraryUsageDescription` keys (see [SETUP.md](#) for exact values)
4. Set minimum deployment to **iOS 17**
5. Run on a real iPhone, tap the **key icon**, paste your Anthropic API key

I installed Xcode to build the app and install on iPhone

- Yes, you need to install Xcode. It's Apple's free IDE for building iOS apps.

Install it from the Mac App Store:

1. Open the **App Store** on your Mac
2. Search for **Xcode**
3. Click **Get** → **Install** (it's free, but large — about 15 GB)
4. Wait for it to download and install (can take 20–60 minutes depending on your connection)

First launch:

- Open Xcode from your Applications folder
- It will ask to install additional components — click **Install** and enter your Mac password
- Accept the license agreement

That's it. Once it's open, follow the steps in [SETUP.md](#) to create the project and add the files.

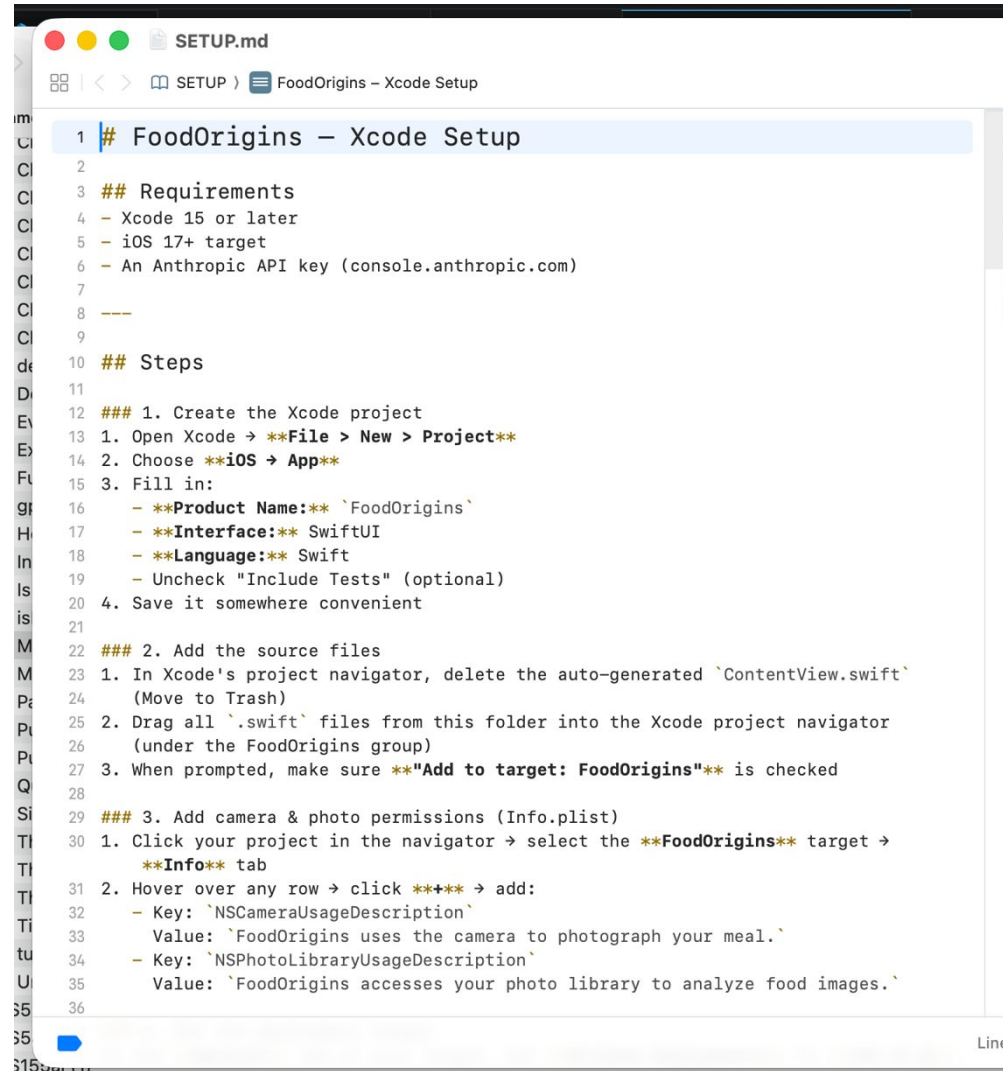
One heads-up: to run the app on your actual iPhone (needed for the camera), you'll also need to:

1. Plug your iPhone into your Mac with a USB cable
2. On your iPhone, tap **Trust** when it asks about the computer
3. In Xcode, select your iPhone from the device dropdown at the top, then press the play button

Ask Claude to edit...

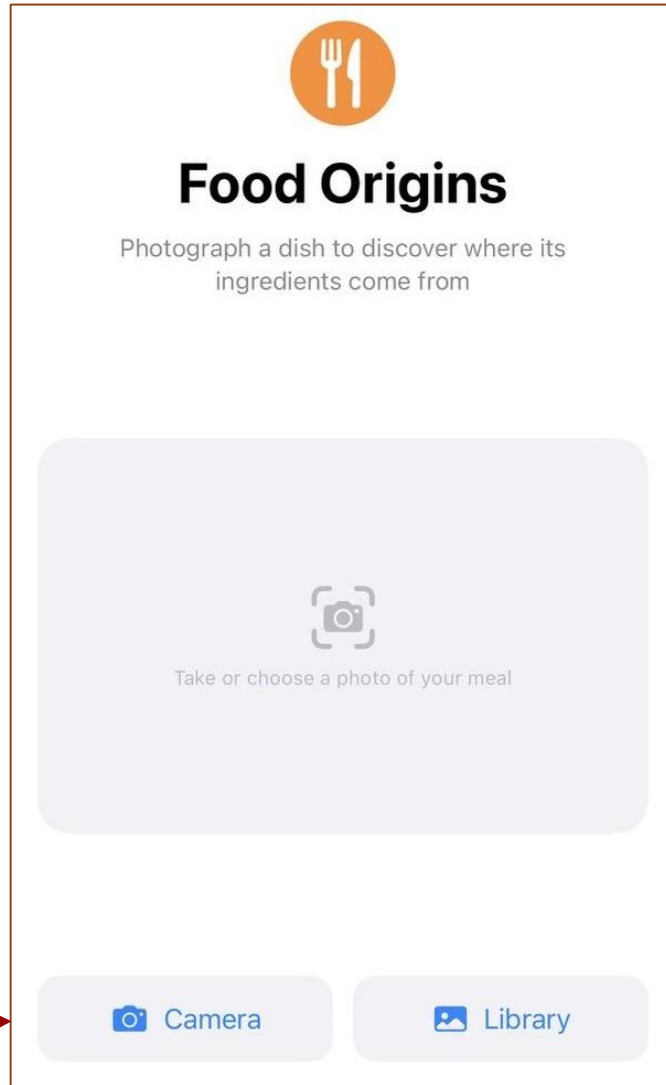


Claude Code wrote instructions for me to use Xcode

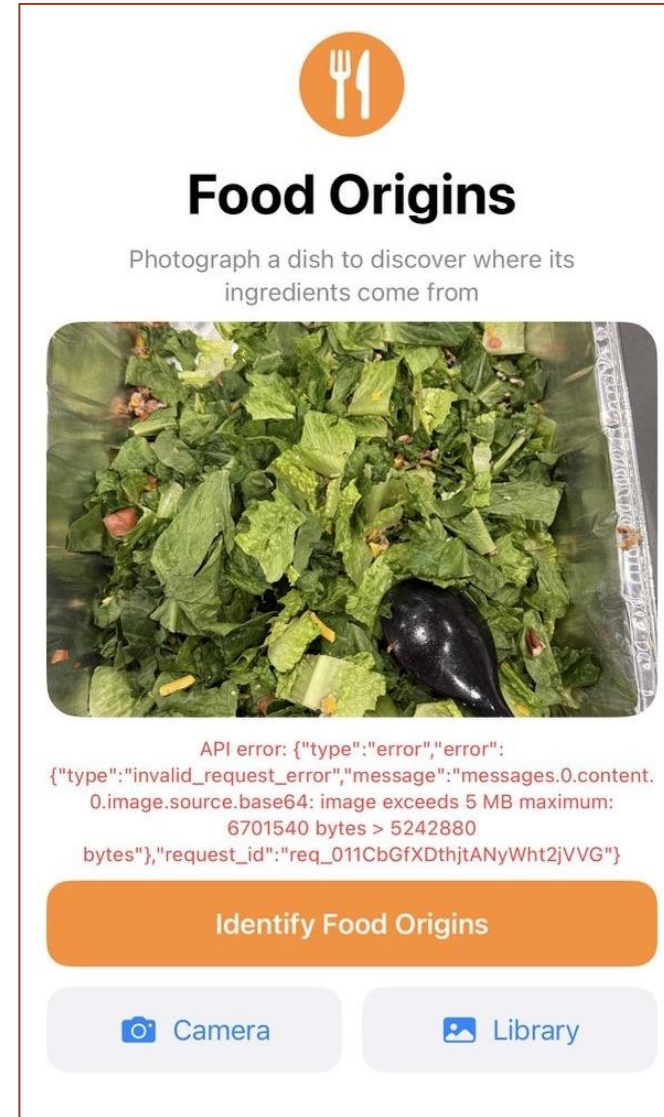


```
1 # FoodOrigins - Xcode Setup
2
3 ## Requirements
4 - Xcode 15 or later
5 - iOS 17+ target
6 - An Anthropic API key (console.anthropic.com)
7
8 ---
9
10 ## Steps
11
12 ### 1. Create the Xcode project
13 1. Open Xcode → File > New > Project
14 2. Choose iOS → App
15 3. Fill in:
16   - Product Name: `FoodOrigins`
17   - Interface: SwiftUI
18   - Language: Swift
19   - Uncheck "Include Tests" (optional)
20 4. Save it somewhere convenient
21
22 ### 2. Add the source files
23 1. In Xcode's project navigator, delete the auto-generated `ContentView.swift`
24   (Move to Trash)
25 2. Drag all `.swift` files from this folder into the Xcode project navigator
26   (under the FoodOrigins group)
27 3. When prompted, make sure "Add to target: FoodOrigins" is checked
28
29 ### 3. Add camera & photo permissions (Info.plist)
30 1. Click your project in the navigator → select the FoodOrigins target →
31   Info tab
32 2. Hover over any row → click +++ → add:
33   - Key: `NSCameraUsageDescription`
34     Value: `FoodOrigins uses the camera to photograph your meal.`
35   - Key: `NSPhotoLibraryUsageDescription`
36     Value: `FoodOrigins accesses your photo library to analyze food images.`
```


The original code produced by Claude Code has bugs



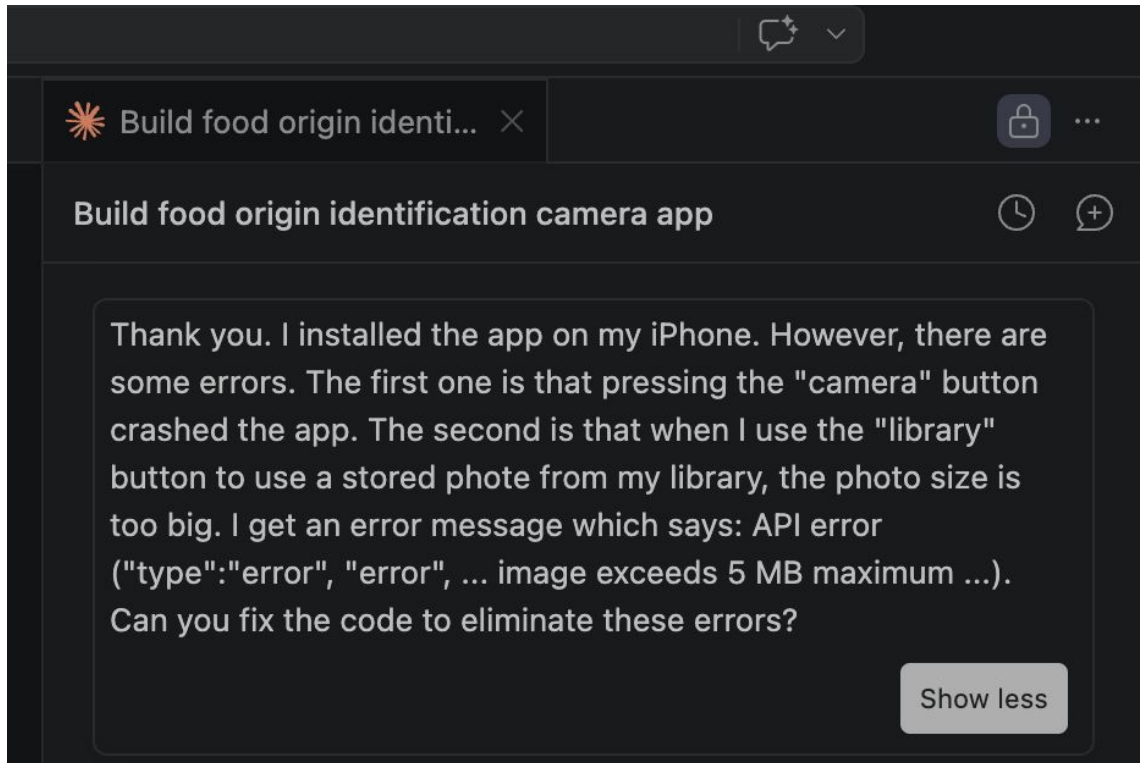
This button
does not work



Library pictures
are too big



Explain the bugs so Claude Code can fix them



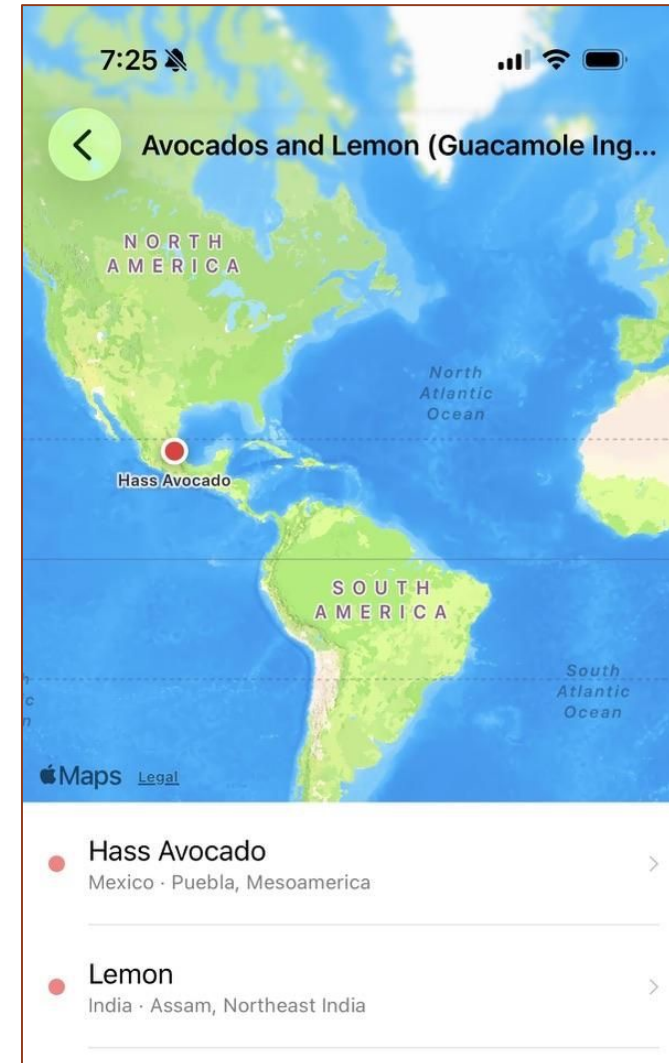
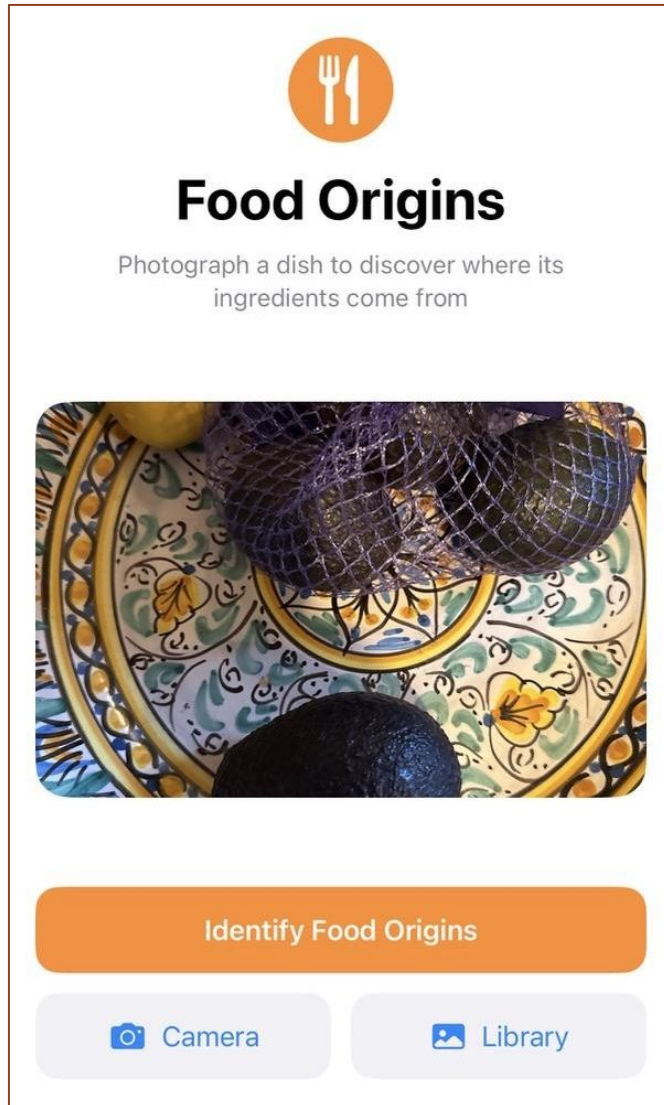
- Both code fixes are done. But the camera **will still crash** without the Info.plist key — iOS enforces this at the OS level before any Swift code runs. Here's how to add it:

Fix the camera crash — add the privacy key in Xcode:

1. In Xcode, click your **FoodOrigins** project in the left sidebar (the blue icon at the top)
2. Select the **FoodOrigins** target (under "Targets")
3. Click the **Info** tab
4. Hover over any row → click the **+** button that appears
5. Type `NSCameraUsageDescription` and press Enter
6. Set the value to: `FoodOrigins needs camera access to photograph your meal.`

Then rebuild and run (`Cmd+R`). The camera button will now prompt for permission on first tap instead of crashing, and library photos will be automatically resized before being sent to Claude.

Wow! It actually works!!



Summary

Prompting

- Ways to query a model in a specific application
- Can be used in combination with other architectures

Retrieval-augmented generation (RAG)

- Retrieve relevant data to augment prompt
- Add information missing from training set, improve accuracy

Agentic AI

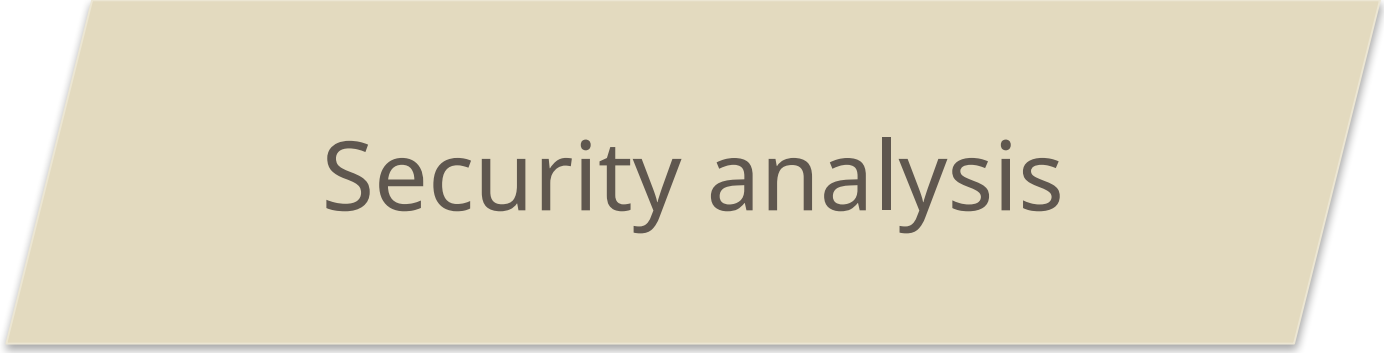
- Structured AI workflow for better functionality than model alone
- Agents can plan, access tools, reflect on workflow, collaborate

Vibe coding

- Ask an agent to build, refactor, extend, ... software for you

Prompt optimization

- Improve accuracy and efficiency by iteratively refining prompts
- Applicable to both a system of interest and agents evaluating it



Security analysis

Security modeling and analysis

A system of interest

Desirable properties of the system

- Usually: the intended purpose or function
- Often: scaled back to most essential user protection

Interface and capabilities of an attacker

- Also called: threat model

Examine whether the attacker can cause the desirable properties of the system to fail

Features

The Science of Security

16 Guest Editors' Introduction

DAVID EVANS AND SAL STOLFO

18 Security Modeling and Analysis

JASON BAU AND JOHN C. MITCHELL

A uniform conceptual framework that precisely defines system security will help analyze security, support comparative evaluation, and develop useful insight into design, implementation, and deployment decisions.

26 On Adversary Models and Compositional Security

ANUPAM DATTA, JASON FRANKLIN, DEEPAK GARG, LIMIN JIA, AND DILSUN KAYNAR

A unified view of a wide range of adversary classes and composition principles for reasoning about security properties of systems are cornerstones of a science of security. They provide a systematic basis for security analysis by explaining and predicting attacks on systems.



COVER ARTWORK BY GIACOMO MARCHESI, WWW.GIACOMOMARCHESI.COM

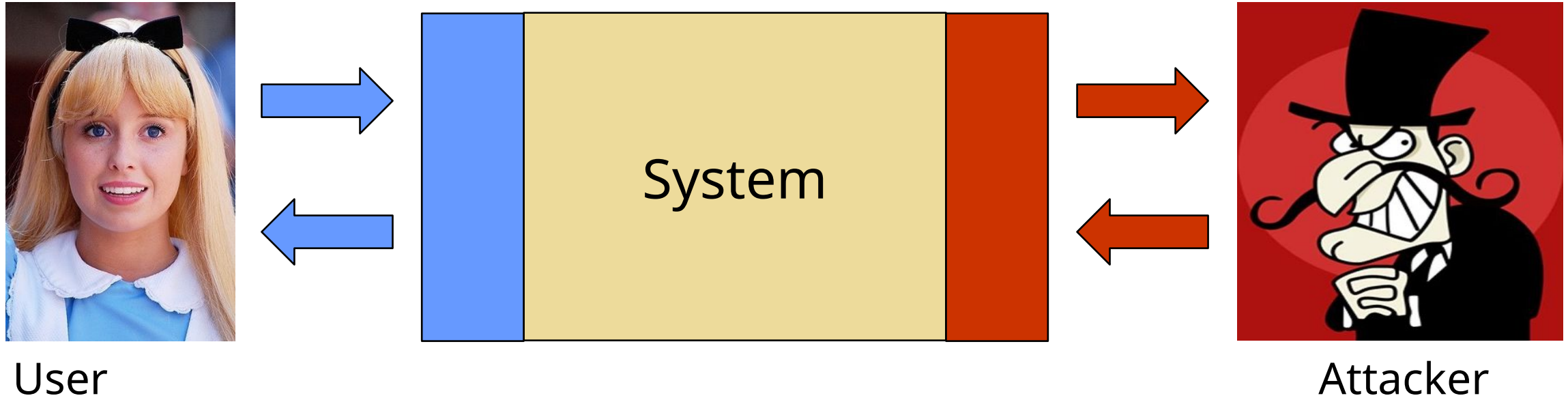
Smartphone Security

42 Secure Software Installation on Smartphones

DAVID BARRERA AND PAUL VAN OORSCHOT

This overview of iOS, Android, BlackBerry, and Symbian security frameworks includes a

System Security



Users interact with the system

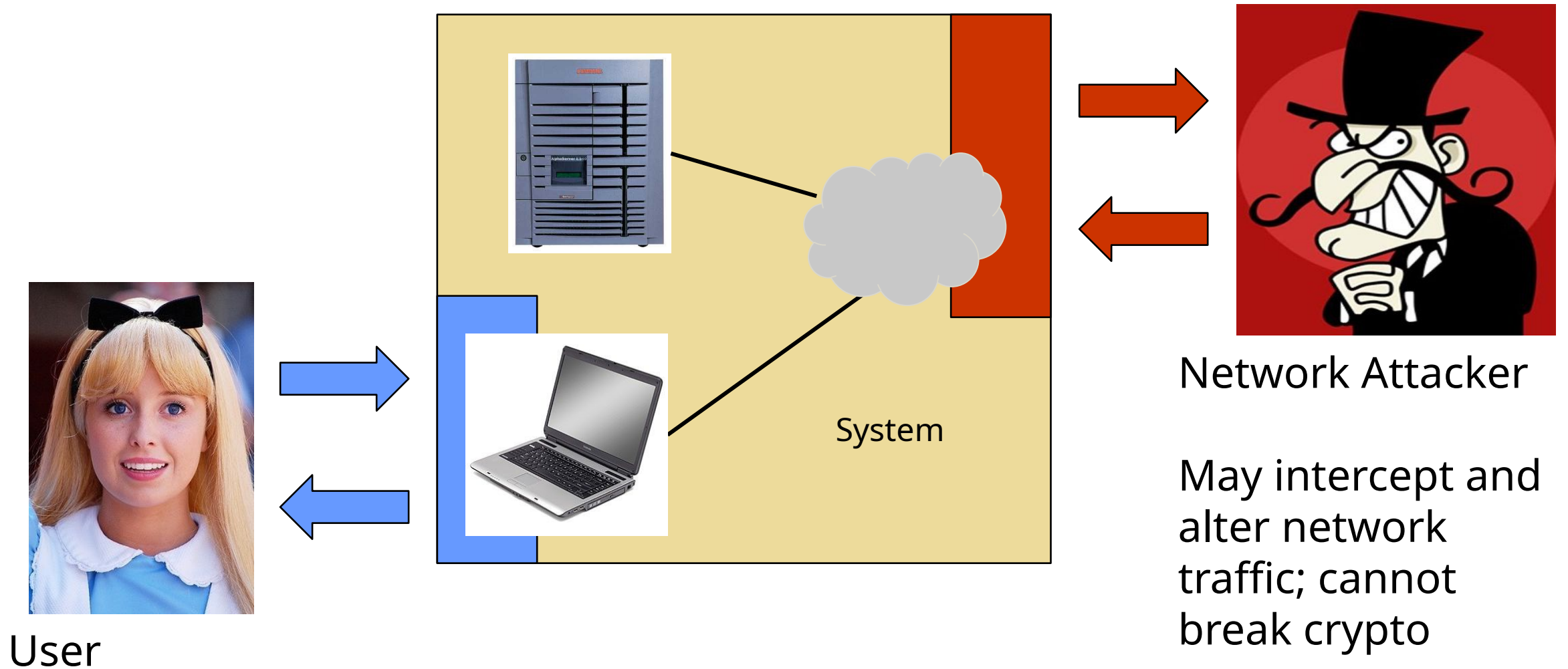
System is intended to serve them in some way

- If users enter expected input, system will deliver desired output

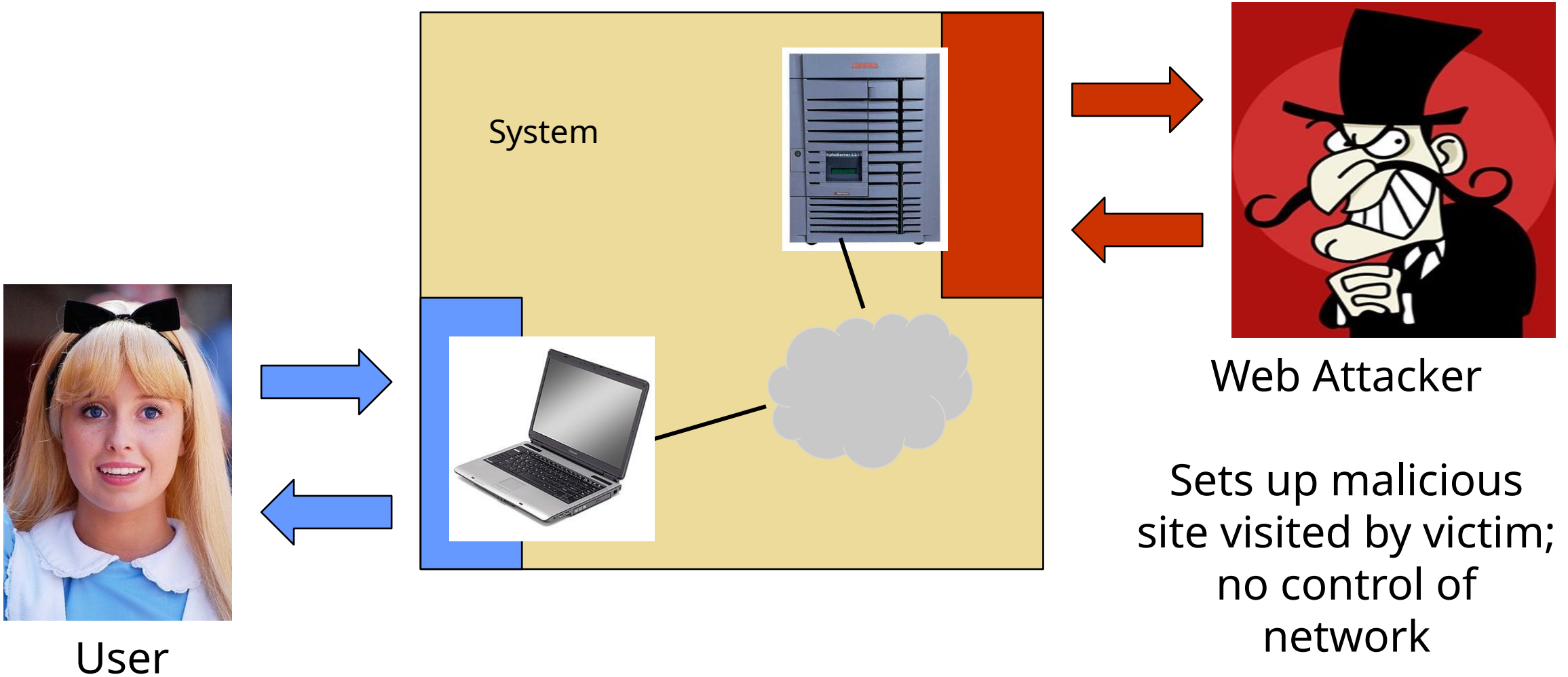
Attacker may have a different interface and capabilities. May:

- Disrupt honest user's use of the system
- Learn information intended for others only

Network Security



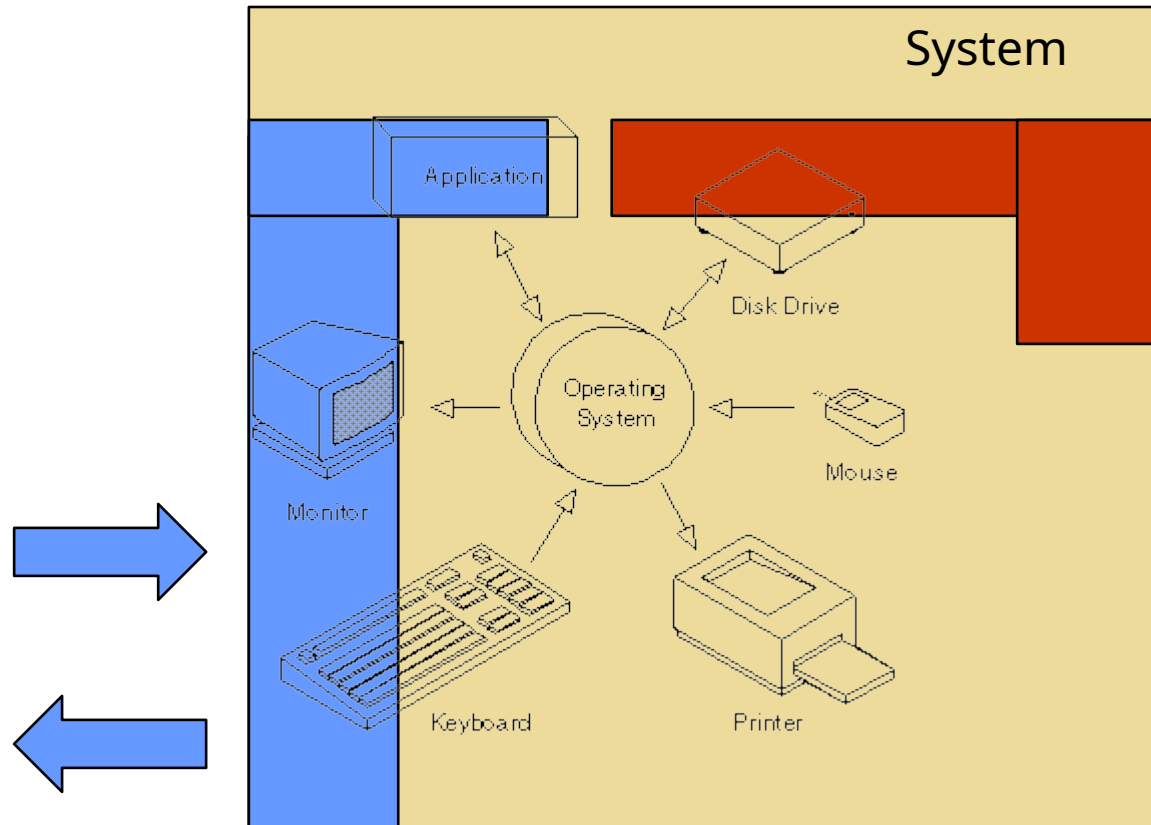
Web Security



Operating System Security



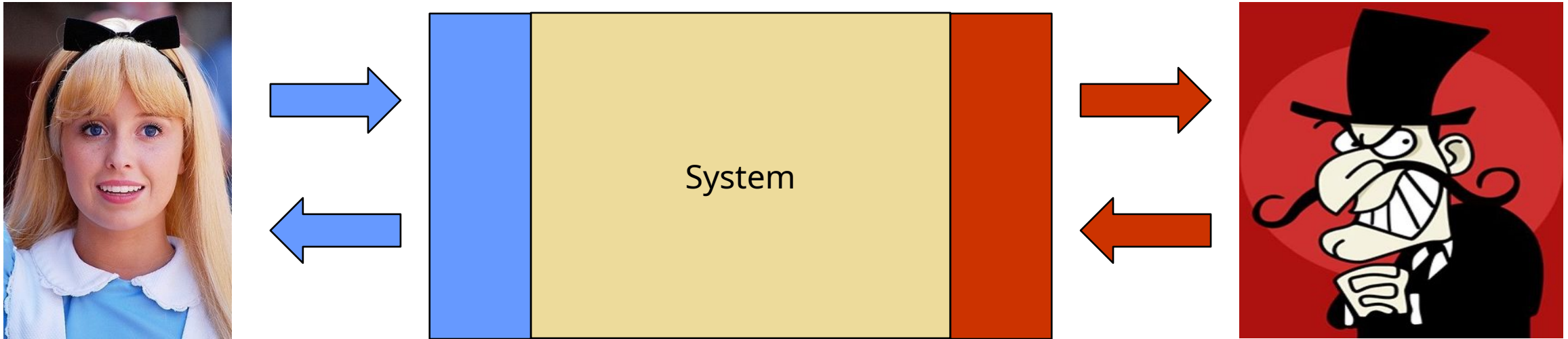
User



OS Attacker

May control
malicious files and
applications

Traditional properties



- Confidentiality
Attacker does not learn Alice's secrets
- Integrity
Attacker does not corrupt system's function for Alice
- Availability
Attacker does not keep system from providing service to Alice

Not an orthogonal decomposition; not necessarily comprehensive